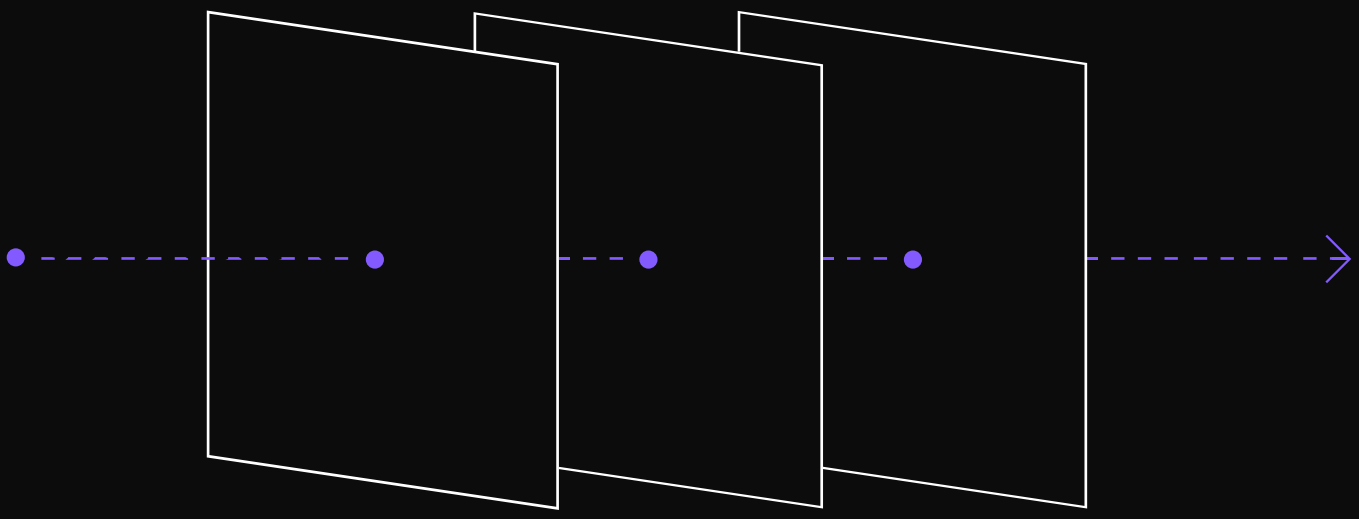
Машинное обучение

Лонгрид

Неделя 1

Метрики и функционалы потерь в задачах регрессии и классификации



Содержание

1. **Machine learning lifecycle (рекап)** 4
2. **Метрика vs функционал потерь** 5
3. **Задача регрессии** 6
   - 3.1. Регрессия: функционалы потерь 6
     - 3.1.1. MAE 7
     - 3.1.2. MSE 7
     - 3.1.3. Оптимальные константы 8
     - 3.1.4. Huber loss 10
     - 3.1.5. Quantile loss 11
   - 3.2. Как выбрать функционал потерь 13
     - 3.2.1. Когда Huber loss подойдёт лучше других loss 14
     - 3.2.2. Когда quantile loss подойдёт лучше других loss 14
   - 3.3. Регрессия: основные метрики 14
     - 3.3.1. MAE — mean absolute error 14
     - 3.3.2. MSE — mean squared error 15
     - 3.3.3. RMSE — root mean squared error 16
     - 3.3.4. R^2 — коэффициент детерминации 17
     - 3.3.5. MAPE — mean absolute percentage error 18
   - 3.4. Как выбрать метрику 19
   - 3.5. Регрессия: good-to-know — RMSLE, sMAPE, WAPE, MdAPE 21
     - 3.5.1. Когда могут пригодиться good-to-know-метрики 22
4. **Задача классификации** 23
   - 4.1. Классификация: функционалы потерь 23
     - 4.1.1. Log loss 23
     - 4.1.2. Categorical cross-entropy loss 25
     - 4.1.3. Hinge loss 26
   - 4.2. Классификация: основные метрики 28
     - 4.2.1. Confusion matrix 28
     - 4.2.2. Метрики бинарной классификации 30
   - 4.3. Бинарная классификация: основные метрики (рекап) 50
5. **Классификация: обобщение на многоклассовые случаи** 50
   - 5.1. Микро-, макро-, взвешенные усреднения 50
   - 5.2. Hamming score 54
   - 5.3. ROC-кривая в многоклассовом случае 55
     - 5.3.1. Micro-average ROC 55
     - 5.3.2. Macro-average ROC 55
   - 5.4. Выбор ML-метрики и функционала потерь 56
     - 5.4.1. Примеры: как может выглядеть логика подбора ML-метрики и loss function 56
     - 5.4.2. Когда метрика и loss совпадают 56
6. **Если кратко** 57

Метрики и функционалы потерь в задачах регрессии и классификации

Контекст

Метрики и функции потерь — это база машинного обучения, которая помогает понимать качество моделей как во время, так и после процесса обучения. Так, уже в следующих неделях мы будем опираться на них при изучении линейных моделей, деревьев решений и ансамблей.

Применение

Ты получишь набор инструментов для выбора метрик и лоссов под конкретную бизнес-задачу. Например узнаешь, почему в финансовом скоринге ключевой метрикой выбирают recall, а в качестве лосса — log loss. И зачем при прогнозировании спроса в ретейле используют относительные метрики вроде sMAPE или RMSLE, а не абсолютные MAE/RMSE.

Краткий план

- → Machine learning lifecycle (рекап)
- → Метрика vs функционал потерь
- → Задача регрессии
- → Задача классификации
- → Классификация: обобщение на многоклассовые случаи
- → Если кратко

1. Machine learning lifecycle (рекап)

ОПРЕДЕЛЕНИЕ

Жизненный цикл машинного обучения (англ. ML lifecycle) — это последовательность этапов, через которые проходит модель машинного обучения: от постановки задачи до выпуска в production и регулярного обновления.

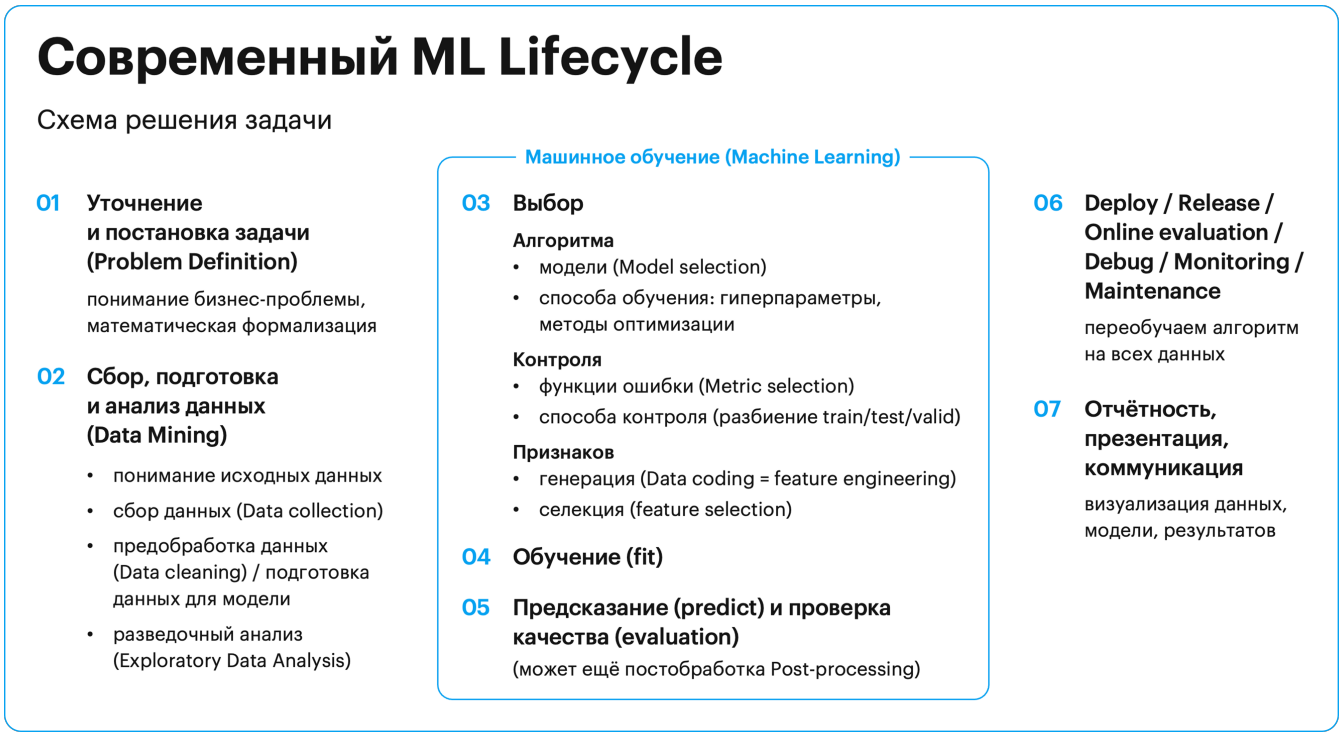


Рис. 1. Семь этапов ML lifecycle

Полный цикл ML-проекта состоит из семи этапов. Сначала — **постановка задачи (problem definition)**: переводим боль бизнеса в конкретную ML-задачу, договариваемся о метриках успеха и проверяем, нужен ли вообще ML. Следом — **data mining**: собираем, чистим и анализируем данные — именно на этот шаг уходит больше всего времени, и именно здесь работает принцип «garbage in — garbage out». Затем выбираем **алгоритм, признаки и способ валидации**, обучаем модель (**fit**) и честно оцениваем её качество (**predict** & **evaluation**) — только на новых данных, только по заранее выбранным метрикам.

Готовое решение **деплоится в продакшен**, после чего начинается мониторинг, поддержка и регулярное переобучение или дообучение, так как данные меняются, а модель устаревает. Финальный, часто недооценённый этап — **презентация и документация** — результаты нужно донести до бизнеса понятным языком.

ВАЖНО

Важно помнить: цикл редко проходится линейно. Плохие результаты на этапе обучения — повод вернуться к данным или постановке задачи, а не искать «волшебный» алгоритм.

На этом курсе изучим современный ML Lifecycle подробнее — поговорим про выбор, обучение и оценку качества ML-моделей. Многие темы из этого курса — фундамент для deep learning.

Основную часть этого лонгрида посвятим базовым понятиям machine learning, а именно machine learning метрикам и функционалам потерь.

2. Метрика vs функционал потерь

ОПРЕДЕЛЕНИЕ

ML-метрика (англ. evaluation metric) — способ количественно оценить качество модели после того, как предсказания уже сделаны.

Метрика отвечает на вопрос: насколько хорошо модель справляется с задачей? Её смотрят при валидации модели, ее тестировании и мониторинге в prod среде.

ОПРЕДЕЛЕНИЕ

Функционал потерь (англ. loss function / loss, по-русски еще говорят «лосс», «функция потерь», иногда говорят «функция ошибки») — формализованная математическая функция, которую модель минимизирует в процессе обучения.

Лосс отвечает на вопрос: как (в каком направлении и насколько сильно) необходимо обновить параметры модели, чтобы качество предсказаний выросло?

ВАЖНО

Многие специалисты путаются в этих терминах, однако их разницу можно описать достаточно просто: **loss — механизм, на основании которого модель учится; метрика — механизм, по которому разработчики оценивают то, насколько хорошо модель обучилась.**

Если совсем упростить определение, функционал потерь — для моделей, ML-метрика — для разработчиков.

Loss и метрика выполняют разные задачи, поэтому и требования разработчиков к ним разнятся.

У **лосса** есть техническое ограничение: он должен быть **дифференцируемым** — то есть в каждой точке должна существовать производная, по которой алгоритм оптимизации понимает, в каком направлении двигаться. Если функция имеет разрывы или «ступеньки», оптимизатор просто не поймёт, что ему делать дальше. Именно поэтому функционалы потерь — математические функции, с которыми просто и удобно работать. Помимо этого, при работе с лоссом разработчики часто смотрят на следующие аспекты.

- **Чувствительность к ошибкам.** Лосс должен реагировать на изменение предсказания — чем хуже предсказание, тем больше значение лосса. Если лосс «не замечает» улучшений, модель не получает полезного сигнала для обучения.
- **Выпуклость** (желательное, но не обязательное свойство).
  - Выпуклая функция имеет один глобальный минимум, и такой оптимизатор как градиентный спуск гарантировано его найдёт.
  - У невыпуклых функций минимумов много, и алгоритм может застрять в локальном. В реальной жизни большинство лоссов невыпуклы, и с этим просто мирятся, используя различные трюки при обучении.
- **Соответствие задаче.** Лосс должен штрафовать именно за те ошибки, которые важны с точки зрения бизнес-контекста. MSE сильно штрафует большие отклонения из-за возведения в квадрат — это хорошо, если крупные ошибки критичны, и плохо, если в данных много выбросов.

Метрики не участвует в процессе оптимизации, поэтому её выбирают по другим критериям.

- **Интерпретируемость.** Метрика должна быть понятна человеку, который принимает решения. RMSE в рублях или MAE в днях — это конкретные числа, которые можно осмыслить. Log loss = 0,43 сам по себе ни о чём не говорит без контекста. Поэтому метрику часто выбирают так, чтобы её можно было объяснить бизнес-аудитории без формул.

- **Связь с бизнес-реальностью.** Хорошая метрика отражает то, что действительно важно в задаче. Если цена ошибки асимметрична — например, пропустить больного пациента гораздо хуже, чем лишний раз его проверить — это должно быть зашито в метрику через выбор recall вместо accuracy. Если бизнес теряет деньги только на крупных ошибках прогноза, а мелкие некритичны — это аргумент в пользу MSE, а не MAE. Метрика — это способ формализовать приоритеты бизнеса в одном числе.
- **Честность оценки.** Метрика должна показывать реальную картину. Accuracy на несбалансированных данных — классический пример «красивой лжи»: модель выглядит отлично на бумаге и бесполезна на практике. Поэтому правильный выбор метрики — это в том числе защита от самообмана.

ВАЖНО

Такие требования делают метрику и функционал потерь взаимодополняемыми частями одного целого. Именно поэтому в практических задачах роль метрики и функционала потерь чаще всего играют разные функции.

Классический пример: модель бинарной классификации обучается по **log loss** (гладкая, дифференцируемая), а оценивается по **accuracy** (дискретная, недифференцируемая). Модель никак не взаимодействует с accuracy во время обучения — она видит только лосс — а accuracy помогает разработчику определить, насколько хорошо модель обучилась.

Разберём все наиболее популярные метрики и функционалы потерь для задач регрессии и классификации. Начнём с задачи регрессии.

3. Задача регрессии

3.1. РЕГРЕССИЯ: ФУНКЦИОНАЛЫ ПОТЕРЬ

Представим универсальные обозначения, которые будем использовать:

y_i — истинное значение таргета для объекта i;

$\hat{y}_i$ — предсказание модели для объекта i;

n — количество наблюдений (объектов в выборке);

$\bar{y}$ — среднее значение таргета по всем объектам.

Здесь важно различать два уровня.

Функция потерь на одном объекте $L(y_i, \hat{y}_i)$ — это штраф за ошибку модели на одном наблюдении. Именно на этом уровне естественно задаются все лоссы ниже — они определяются через один остаток $r_i = y_i - \hat{y}_i$.

Функционал потерь на выборке — то, что модель реально минимизирует при обучении. Это средний штраф по всем объектам:

$$Loss = \frac{1}{n}\sum_{i=1}^{n} L(y_i, \hat{y}_i)$$

В речи лоссом могут называть и то, и другое, но формально это разные вещи, ведь сама функция L живёт на одном объекте, а обучение идёт по её усреднению. Далее в лонгриде для каждого лосса мы сначала выпишем L на объекте, а затем подставим получившееся в формулу выше.

3.1.1. MAE

Самое простое, что можно сделать при сравнении двух массивов чисел — сравнить все элементы попарно по модулю и усреднить результат.

Именно такой подход реализует **MAE (mean absolute error)**:

$$L_{MAE}(y_i, \hat{y}_i) = |y_i - \hat{y}_i|$$

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n} |y_i - \hat{y}_i|.$$

Когда MAE выступает лоссом, модель получает одинаковый штраф за ошибку на 1 и за ошибку на 100 — в пересчёте на единицу отклонения. Это делает обучение устойчивым к выбросам: один аномальный объект не перетянет на себя всё внимание модели.

ПРИМЕР

- Ошибка увеличилась с 0 до 1 $\Longrightarrow$ штраф +1.
- Ошибка увеличилась с 99 до 100 $\Longrightarrow$ штраф тоже +1.

MAE одинаково относится к любому увеличению ошибки.

Но у MAE как у функции потерь есть математический изъян — производная модуля не определена в нуле. В точке, где предсказание совпадает с истинным значением, градиент просто не существует. На практике это решают заменой на субградиент, но глобально градиентные методы оптимизируют MAE заметно хуже, чем тот же MSE.

MAE как лосс хорошо подходит в следующих случаях.

- В данных есть выбросы, которые не должны доминировать при обучении.
- Все ошибки равнозначны по бизнес-смыслу.
- Важна интерпретируемость: модель обучается минимизировать среднее абсолютное отклонение — ровно то, что затем измерит метрика MAE.

3.1.2. MSE

MSE (mean squared error) — одна из самых популярных функций потерь в задачах регрессии:

$$L_{MSE}(y_i, \hat{y}_i) = (y_i - \hat{y}_i)^2$$

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2.$$

Возведение в квадрат делает две вещи одновременно: избавляет от знака (как модуль в MAE) и усиливает штраф за крупные ошибки, например, за счёт возведения в степень ошибка 10 штрафуется в 100 раз сильнее, чем ошибка 1.

ПРИМЕР

- Ошибка увеличилась с 0 до 1 $\Longrightarrow$ штраф: $1^2 - 0^2 = 1$.
- Ошибка увеличилась с 99 до 100 $\Longrightarrow$ штраф: $100^2 - 99^2 = 199$.

По сравнению с MAE, MSE намного штрафует большие ошибки гораздо сильнее.

Как функция потерь MSE имеет важное преимущество перед MAE: она гладкая и дифференцируема везде, включая ноль. Градиент MSE вычисляется просто, что делает эту функцию удобной для градиентного спуска.

Обратная сторона — чувствительность к выбросам. Один объект с большой ошибкой квадратично увеличивает лосс и тянет модель в свою сторону. Если выбросы в данных случайные и не несут смысла, это проблема. Если выбросы важны (например, крупные кредитные риски) — это фича.

MSE как лосс хорошо подходит в следующих случаях.

- Важно минимизировать крупные ошибки, даже ценой увеличения мелких.
- Данные относительно чистые, без случайных аномалий.
- В качестве оптимизатора используется градиентный спуск.

3.1.3. Оптимальные константы

Есть элегантный способ понять разницу между MAE и MSE на интуитивном уровне — спросить, какое одно число лучше всего «представляет» все объекты выборки при каждом из лоссов.

Представим, что у нас нет никаких признаков — только таргет. Модель вынуждена предсказывать одно и то же значение c для всех объектов. Какое c минимизирует лосс?

ОПРЕДЕЛЕНИЕ

Оптимальная константа — это наилучшее предсказание, которое модель может сделать, если не знает ничего о признаках объекта и вынуждена предсказывать одно и то же значение для всех наблюдений.

Формально, оптимальная константа это такое значение c , которое минимизирует выбранную функцию потерь на обучающей выборке при условии, что предсказание константно:

$$\text{c} = \text{argmin}_c \sum_{i=1}^{n} \text{L}(\text{y}_i, \text{c}).$$

ВАЖНО

На практике оптимальная константа используется как **бейзлайн** — нижняя планка, ниже которой модель опускаться не должна. Если обученная модель работает хуже оптимальной константы, что-то пошло серьёзно не так: либо с данными, либо с обучением.

Оптимальная константа MSE стремится к среднему таргета. Минимизируем:

$$\frac{1}{n}\sum_{i=1}^{n}(y_i - c)^2.$$

Берём производную по c, приравниваем к нулю:

$$\frac{d}{dc}\sum(y_i - c)^2 = -2\sum(y_i - c) = 0;$$

$$c = \frac{1}{n}\sum y_i = \bar{y}.$$

Оптимальная константа для MSE — это **среднее** значение таргета.

В свою очередь оптимальная константа MAE стремится к медиане таргета. Минимизируем:

$$\frac{1}{n}\sum_{i=1}^{n}|y_i - c|.$$

Производная модуля — это знаковая функция: $\text{sign}(y_i - c)$. Сумма обращается в ноль, когда ровно половина объектов лежит выше c, а вторая половина ниже — то есть когда c является **медианой**.

ВАЖНО

Это не просто математический факт — это ключ к пониманию поведения лосс функций. MSE тянет предсказание к среднему, которое чувствительно к выбросам. MAE тянет к медиане, которая является более устойчивым значением. Именно поэтому выбор лосса определяет то, что именно модель учится предсказывать.

ПРИМЕР

Представим, что модель может предсказывать только одно постоянное число c для всех объектов. Для выборки:

$$[1, 2, 3, 4].$$

И для MSE, и для MAE оптимальный c будет в районе:

$$c \approx 2,5.$$

Но что произойдёт, если в выборке будет явный выброс:

$$[1, 2, 3, 4, 100].$$

В случае MSE, если предсказание модели случится около нормальных данных, например $c = 3$, то ошибка на выбросе станет огромной:

$$(100 - 3)^2 = 9409.$$

Для сравнения, ошибка для обычной точки будет:

$$(4 - 3)^2 = 1.$$

То есть для MSE один выброс даст такой же вклад, как и тысячи обычных ошибок. Именно поэтому MSE выгоднее пожертвовать нормальными точками и сдвинуть оптимальную константу ближе к выбросу:

$$c \approx 22.$$

Так квадрат ошибки выброса уменьшится, и суммарный MSE станет меньше.

Для MAE ошибка считается без квадрата. При $c = 3$:

$$|100 - 3| = 97.$$

А для обычной точки:

$$|4 - 3| = 1.$$

Выброс всё ещё оказывает сильное влияние, но только линейно, а не квадратично. Поэтому модели невыгодно сильно двигать предсказание к 100, и оптимальное значение почти не изменится меняется:

$$c = 3.$$

Этот пример наглядно демонстрирует следующее.

- **MSE чувствителен к выбросам** и тянется к ним.
- **MAE устойчив к выбросам** и остаётся ближе к основной массе данных.

3.1.4. Huber loss

MAE устойчива к выбросам, но плохо оптимизируется. MSE хорошо оптимизируется, но чувствительна к выбросам. **Huber loss** — попытка взять лучшее от обеих.

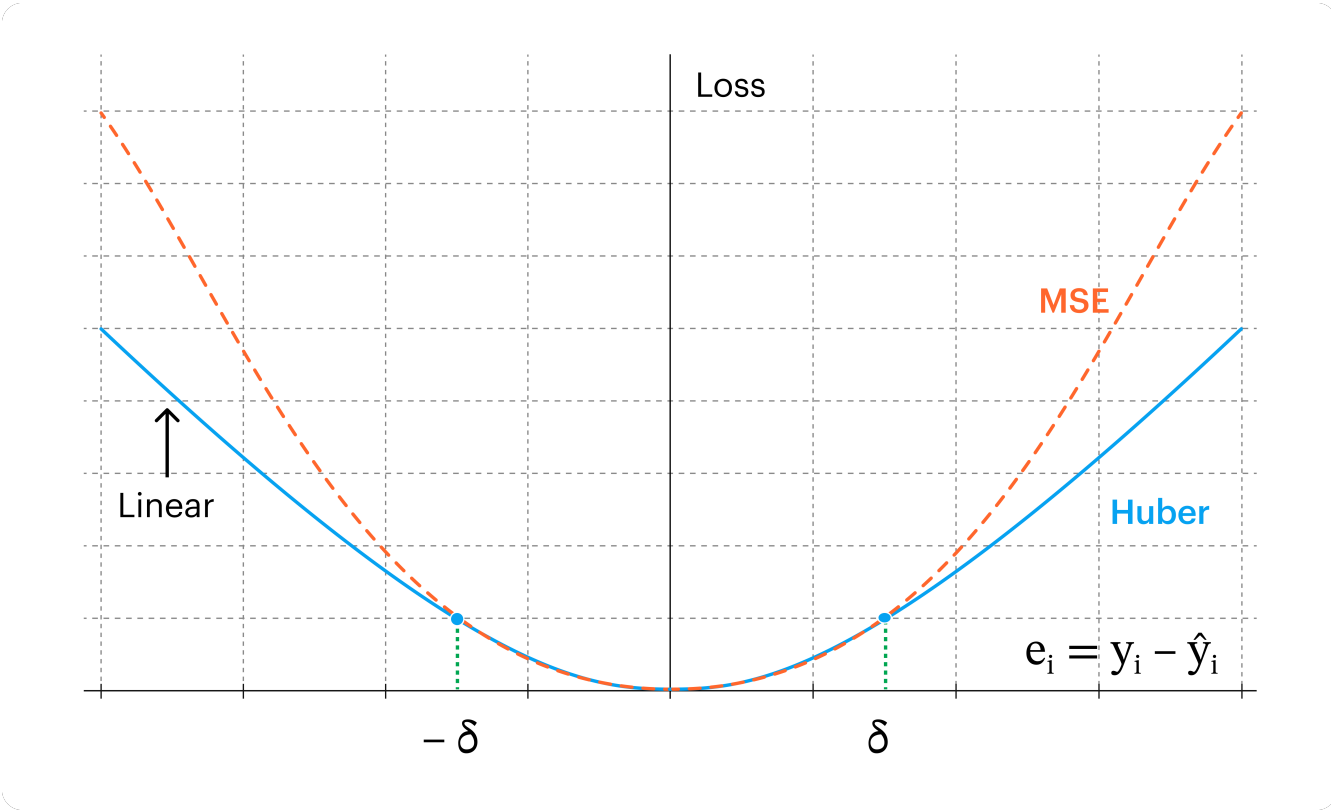


Рис. 2. Визуализация графика Huber loss

Идея простая: для небольших ошибок вести себя как MSE (гладко, с сильным градиентом), а для больших — как MAE (линейно, без квадратичного усиления). Граница между режимами задаётся гиперпараметром δ.

$$L_\delta(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2, & |y - \hat{y}| \le \delta \\ \delta \cdot |y - \hat{y}| - \frac{1}{2}\delta^2, & |y - \hat{y}| > \delta \end{cases}$$

Как и раньше, это функция потерь на одном объекте. Функционал на выборке — её среднее:

$$\text{Huber} = \frac{1}{n}\sum_{i=1}^{n} L_\delta(y_i, \hat{y}_i).$$

Huber loss дифференцируема везде, в том числе в точке перехода — это обеспечивает вычитание $\frac{1}{2}\delta^2$ в линейной части.

При работе с Huber loss важно правильно выбрать гиперпараметр δ. Зачастую он подбирается в зависимости от данные: маленькое значение делает функцию ближе к MAE, большое — ближе к MSE.

Huber loss хорошо подходит в следующих случаях.

- Данные содержат выбросы, но полностью терять чувствительность к крупным ошибкам нежелательно.
- Важна стабильность обучения при наличии аномалий.
- Есть возможность подобрать δ на основе распределения ошибок в данных.

ВАЖНО

Оптимальная константа Huber loss также зависит от δ. При маленьком δ функция ведёт себя почти как MAE, и константа близка к медиане. При большом δ — почти как MSE, а константа близка к среднему. В общем случае это усечённое среднее (trimmed mean): среднее, посчитанное без объектов с самыми большими ошибками. Huber loss как бы говорит: «считай среднее, игнорируя выбросы».

3.1.5. Quantile loss

MAE, MSE и Huber loss объединяет одна цель: минимизировать ошибку симметрично — переоценка и недооценка штрафуются одинаково. Но во многих бизнес-задачах цена ошибки асимметрична.

Например: склад заказывает товар на основе прогноза спроса. Заказать слишком мало — дефицит и потеря продаж. Заказать слишком много — замороженный капитал и хранение. Эти потери неодинаковы. Модель, которая ошибается симметрично, может не отражать реальных бизнес-приоритетов.

Quantile loss (он же pinball loss) решает эту задачу, позволяя явно задать асимметрию штрафов через параметр квантили $\tau \in (0, 1)$. С интерактивной визуализацией этого функционала потерь можно ознакомиться здесь.

$$L_\tau(y, \hat{y}) = \begin{cases} \tau \cdot (y - \hat{y}), & y \ge \hat{y} \\ (1 - \tau) \cdot (\hat{y} - y), & y < \hat{y} \end{cases}$$

Функционал на выборке:

$$\text{Quantile} = \frac{1}{n}\sum_{i=1}^{n} L_\tau(y_i, \hat{y}_i).$$

При $\tau = 0,5$ штрафы симметричны — это MAE (с точностью до константы). При $\tau = 0,9$ недооценка штрафуется в 9 раз сильнее переоценки: модель учится предсказывать консервативно, с запасом вверх.

Так же как MSE тянет предсказание к среднему, а MAE — к медиане, quantile loss тянет предсказание к квантили τ распределения таргета.

Quantile loss хорошо подходит в следующих случаях.

- Цена ошибки в разные стороны различается (асимметричные издержки).
- Нужно предсказывать не среднее, а конкретную квантиль распределения.

ВАЖНО

Оптимальная константа для quantile loss это квантиль τ распределения таргета. При $\tau = 0,9$ — это 90-й перцентиль выборки. Это прямое следствие того, о чём уже говорили: quantile loss целенаправленно тянет предсказание к нужной квантили.

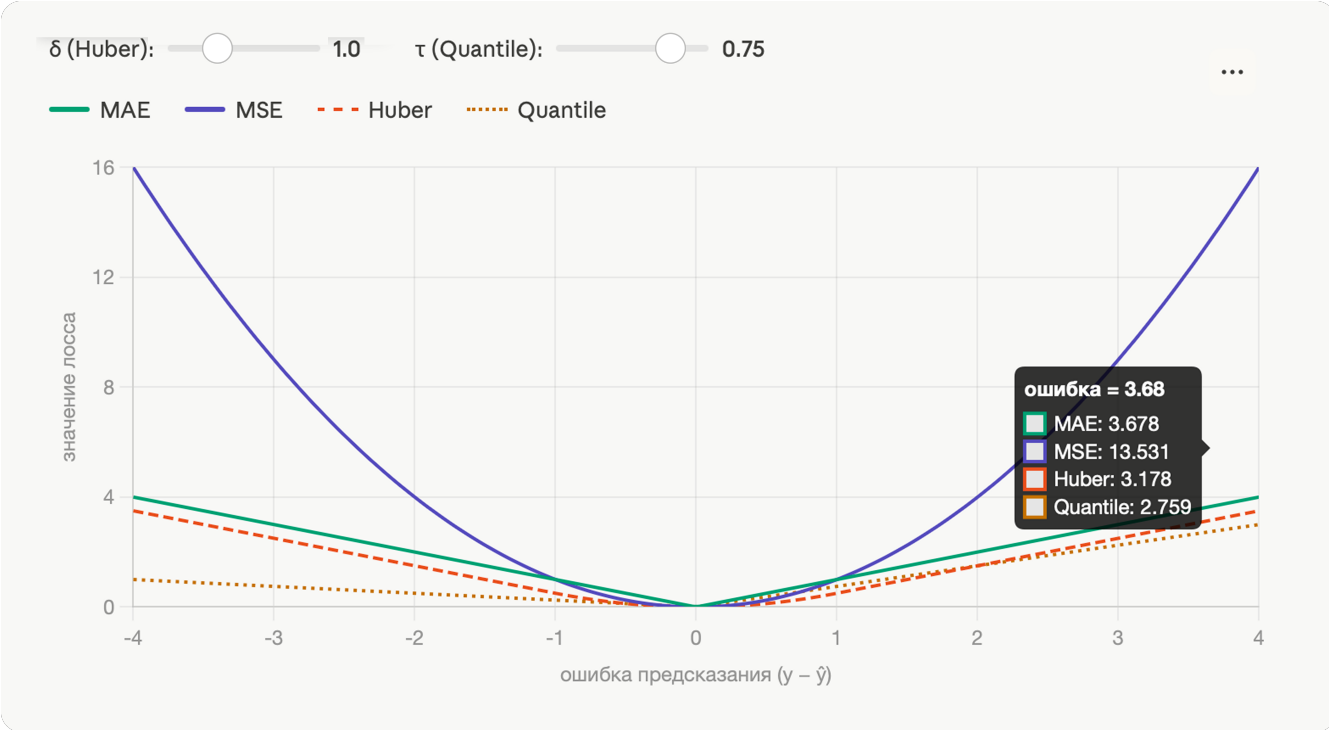


Рис. 3. Визуализация штрафов за ошибки разного размера для loss-функций — MAE, MSE, Huber loss, quantile loss. Сгенерировано через Claude

3.2. КАК ВЫБРАТЬ ФУНКЦИОНАЛ ПОТЕРЬ

Loss	Что минимизирует	К чему тянет предсказание	Чувствительность к выбросам	Плюсы	Минусы	Когда использовать
MAE	Среднюю абсолютную ошибку	К медиане	Низкая	Устойчива к выбросам, интерпретируема	Плохо оптимизируется градиентно	Шумные данные, аномалии, равная цена всех ошибок
MSE	Средний квадрат ошибки	К среднему	Очень высокая	Гладкая, удобна для оптимизации	Выбросы могут доминировать	Чистые данные, важно сильно штрафовать крупные ошибки
Huber loss	Компромисс MAE/MSE	Между медианой и средним	Средняя	Стабильное обучение + устойчивость	Нужно подбирать δ	Реальные данные с умеренным количеством выбросов
Quantile loss	Асимметричную ошибку	К выбранному квантилю	Зависит от τ	Учитывает бизнес-стоимость ошибок	Менее интуитивна	Forecasting, risk management, supply chain

Главное о чем стоит помнить при выборе loss функции это то, что она определяет не просто то как обучается модель, а то, **что именно модель считает хорошим предсказанием.**

В случае с MSE и MAE разобраться в этом довольно просто.

- MSE говорит: «любой ценой не допускай больших ошибок».
- MAE говорит: «ошибайся стабильно и не реагируй на выбросы».

Понять, когда стоит применить Huber loss или quantile loss, немного сложнее.

- Quantile loss говорит: «ошибайся в нужную бизнесу сторону».
- Huber loss говорит: «старайся не реагировать на выбросы, но при этом не теряй чувствительности к крупным ошибкам».

3.2.1. Когда Huber loss подойдёт лучше других loss

Huber loss стремится обучаться точно на нормальных объектах и не ломать всё обучение из-за случайного большого выброса.

Huber loss часто выигрывает на реальных табличных данных:

- где выбросы есть, но их немного;
- где нужен стабильный градиентный спуск;
- когда MAE слишком нестабилен, а MSE слишком чувствителен.

Другими словами, Huber — очень практичная функция потерь для noisy regression.

ПРИМЕР

- кредитный скоринг,
- прогноз спроса,
- предсказание выручки,
- сенсорные данные IoT.

3.2.2. Когда quantile loss подойдёт лучше других loss

Quantile loss позволяет модели специально завышать или занижать прогноз, если одна из ошибок стоит бизнесу дороже.

Quantile loss особенно силён:

- когда цена недооценки и переоценки различается;
- нужно учиться не на среднем значении, а на нужной бизнесу границе риска.

3.3. РЕГРЕССИЯ: ОСНОВНЫЕ МЕТРИКИ

Ниже представлено систематизированное описание метрик регрессии в виде структурированной шпаргалки, к которой можно обратится, чтобы освежить знания.

3.3.1. MAE — mean absolute error

ОПРЕДЕЛЕНИЕ

MAE (mean absolute error) — метрика качества модели в задачах регрессии, которая измеряет, насколько в среднем предсказания модели отличаются от реальных значений:

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|.$$

Интерпретация результата проста.

- MAE = 0 — предсказания идеальны.
- Чем меньше MAE, тем лучше модель.
- Единицы измерения MAE такие же, как у таргета (например, если модель предсказывает цену в € —

MAE тоже в €).

- MAE = 5 означает, что в среднем модель ошибается на 5 единиц.

Всё это делает MAE простой, интерпретируемой и устойчивой к выбросам метрикой, но за этими преимуществами есть ряд серьёзных недостатков:

- MAE не учитывает «силу ошибок»: ошибки на 1 и 100 влияют одинаково (в отличие от других метрик, которые мы затронем позже).
- MAE не показывает «занижает» или «завышает» модель предсказания.

Но бывают случаи, когда недостатки не столь существенны, и MAE в качестве ML-метрики можно эффективно использовать. Например:

- когда все ошибки равны по значимости и выбросы допустимы (например, предсказание температуры, стоимости товара, веса и так далее);
- если бизнес-задача требует прогнозировать точно в среднем, без особого акцента на крупные промахи;
- когда важна простота интерпретации.

3.3.2. MSE — mean squared error

ОПРЕДЕЛЕНИЕ

MSE (mean squared error) — это одна из самых популярных метрик в задачах регрессии, которая измеряет среднюю величину квадрата отклонения предсказания от истинного значения:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2.$$

Интерпретация MSE схожа с MAE.

- MSE = 0 — предсказания идеальны.
- Чем меньше MSE, тем лучше модель.
- Поскольку ошибки возводятся в квадрат, большие ошибки штрафуются сильнее (оказывают большее влияние на итоговое значение метрики).
- Единицы измерения метрики оказываются в квадрате относительно таргета (например, если таргет в € → MSE в $€^2$), что усложняет интерпретацию.

Преимущества MSE:

- бóльшие ошибки штрафуются сильнее, что важно в задачах, где большие отклонения критичны (например, в медицине или в ценообразовании).

Недостатки MSE:

- MSE слишком чувствительна к выбросам — одна крупная ошибка может сильно исказить общую картину;
- MSE хуже интерпретируется, так как из-за возведения в степень бывает сложно сказать, «насколько в среднем ошибается модель» в привычных нам величинах.

Тем не менее, MSE широко применяется и как ML-метрика, особенно в следующих случаях:

- важно минимизировать крупные ошибки, даже ценой увеличения небольших;
- выбросы важны для бизнеса (например, в предсказании кредитного риска или прогноза спроса);
- используются модели, которые обучаются с помощью градиентного спуска — MSE хорошо оптимизируется.

3.3.3. RMSE — root mean squared error

Недостаток MSE в виде квадратирования единиц измерения можно легко исправить.

ОПРЕДЕЛЕНИЕ

RMSE (root mean squared error) — это метрика качества модели в задачах регрессии, которая показывает, насколько сильно в среднем предсказания модели отклоняются от фактических значений, при этом сильно наказывая большие ошибки, как и MSE.

С точки зрения формулы RMSE — просто корень из MSE:

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}.$$

Интерпретация RMSE схожа с MSE.

- RMSE = 0 означает идеальные предсказания.
- Значение выражается в тех же единицах, что и целевая переменная — это ключевое отличие MSE от RMSE.
- RMSE показывает среднюю величину ошибки, но при этом больше «боится» больших промахов (так как берётся квадрат перед корнем).

RMSE чаще используется в отчётах для бизнеса, так как она интуитивно более понятна, чем MSE, но при этом всё равно штрафует большие ошибки.

- Однако несмотря на корень, RMSE менее устойчива к выбросам, чем MAE.
- Как и MAE, RMSE не показывает распределение ошибок. Например, RMSE = 100 € не позволяет понять, связано ли это со множеством мелких ошибок или одной очень крупной.

RMSE — удобный компромисс между MAE и MSE. Эту метрики применяют, когда важно минимизировать крупные ошибки, но при этом нужно получить понятное измерение для презентации результатов.

ВАЖНО

Все метрики / функционалы потерь, которые мы обсудили выше, дают только абсолютные числа — средние абсолютные/квадратичные ошибки.

Их значения требуют интерпретации и контекста и не могут в вакууме ответить на вопросы «Насколько хорошей получилась модель?», «Лучше ли она модели, которая будет просто предсказывать среднее целевой переменной?», «Насколько хорошо модель объясняет данные?».

3.3.4. R^2 — коэффициент детерминации

Чтобы ответить на эти вопросы, разработчики используют R^2.

ОПРЕДЕЛЕНИЕ

Коэффициент детерминации, или **R^2** (читается как «эр квадрат»), — это метрика качества модели в задачах регрессии, которая показывает, насколько сильно в среднем предсказания модели отклоняются от фактических значений, при этом сильно наказывая большие ошибки, как и MSE.

R^2 сравнивает ошибку модели с ошибкой простейшего бейзлайна — модели, которая всегда предсказывает среднее значение по выборке:

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}.$$

- Числитель — среднеквадратичная ошибка (MSE) модели.
- Знаменатель — ошибка самой простой модели, которая всегда возвращает среднее значение $\bar{y}$. Фактически это лучшая константа, которую мы можем подставить, чтобы получить лучшее значение MSE без обучения модели.

Интерпретировать коэффициент детерминации можно следующим образом.

- $R^2 = 1$ — идеальная модель (все точки предсказаны точно).
- $R^2 = 0$ — имеющаяся модель не лучше той, которая просто предсказывает среднее значение по выборке.
- $R^2 < 0$ — имеющаяся модель хуже бейзлайна (ошибки больше, чем у «модели-среднего»).

Часто можно услышать, что R^2 показывает, какую долю дисперсии целевой переменной объясняет модель. На практике это означает следующее.

ВАЖНО

R^2 = 0,82 — 82% всех колебаний (изменений) в целевой переменной модель смогла «поймать» своими предсказаниями. То есть модель поняла 82 из 100 причин, почему целевая переменная изменяется.

Остальные 18% изменений модель не смогла объяснить: это шум, случайные колебания, скрытые факторы и ошибки.

R^2 — удобная и интуитивно понятная метрика, показывающая, насколько ошибка модели меньше ошибки простейшего бейзлайна, предсказывающего среднее значение таргета. Значения ближе к 1 означают лучшее объяснение вариации данных моделью

Но у коэффициента детерминации есть ряд недостатков.

- R^2 может быть отрицательным, и тогда не всегда очевидно, что он означает.
- R^2 не устойчив к выбросам, как и любая метрика, основанная на MSE.
- R^2 может вводить в заблуждение.
  - Высокий R^2 не гарантирует, что модель хороша (особенно если она смещена, систематически занижает/завышает предсказания).
  - В задачах с несбалансированными данными или нелинейными зависимостями R^2 может быть плохо интерпретируем.

- Зависит от диапазона и дисперсии целевой переменной: в случаях, когда дисперсия целевой переменной мала, даже хорошая модель покажет низкий R^2.

Обрати внимание на R^2, если работаешь:

- с традиционными регрессионными задачами, где важна интерпретация в стиле — насколько хорошо модель объясняет поведение целевой переменной;
- задачей, где нужно сравнить модель с бейзлайном или несколько моделей между собой.

3.3.5. MAPE — mean absolute percentage error

Одна из самых интерпретируемых метрик в задаче регрессии — это MAPE.

ОПРЕДЕЛЕНИЕ

MAPE (mean absolute percentage error) — это метрика, которая измеряет среднюю относительную (в процентах) ошибку модели в задаче регрессии:

$$\text{MAPE} = \frac{100\%}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right|.$$

Результат MAPE показывает, на сколько процентов в среднем предсказания модели отличаются от истинных значений. Например, MAPE = 12% означает, что в среднем модель ошибается на 12% от реального значения. Чем меньше MAPE — тем лучше.

Подобный концепт дарит MAPE несколько преимуществ.

- MAPE интуитивно понятна — процент ошибочного отклонения легко объяснить, что делает MAPE частой метрикой в докладах бизнесу.
  - В бизнес-статьях можно часто встретить формулировку «ошибка модели находится в пределах 7–10%».
- MAPE не зависит от единиц измерения — так как ответ всегда в процентах, можно сравнивать модели, которые решают разные задачи.

Но стоит помнить, что MAPE:

- не определена, если $y_i = 0$ — деление на ноль. Даже близкие к нулю значения в целевой переменной могут сильно искажать результат;
- переоценивает ошибки при маленьких y_i:
  - ошибка 1 на значении 1 = 100%;
  - ошибка 1 на значении 100 = всего 1%;
  - из-за этой особенности MAPE нестабильна, если в выборке есть маленькие значения и/или широкий диапазон целевой переменной;
- не симметрична — одна и та же абсолютная ошибка будет сильнее «наказана», если произошла при малом значении y_i;
- плохо устойчива к выбросам, особенно если они находятся около нуля.

3.4. КАК ВЫБРАТЬ МЕТРИКУ

Построим график. По оси X отложим значение ошибки $|y - \hat{y}|$, по оси Y — сколько эта ошибка «стоит» каждой метрике. Посмотрим на форму функций для каждой метрики: как быстро растёт штраф при увеличении отклонения.

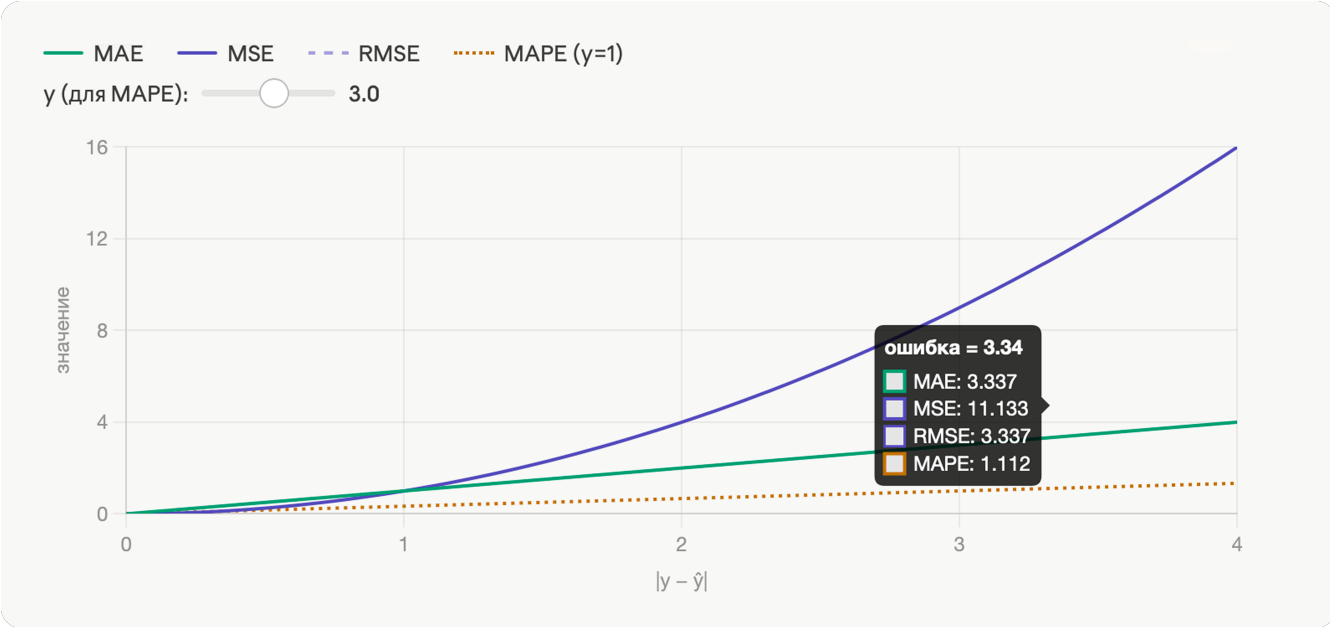


Рис. 4. Значение штрафа метрики в зависимости от размера ошибки. Сгенерировано через Claude

RMSE совпадает с MAE, так как в данном случае мы смотрим, какое значение метрики мы получим, если размер ошибки будет равен X. В такой постановке, значение RMSE будет просто модулем ошибки. Именно поэтому на графике RMSE и MAE неотличимы. Их разница проявляется только когда в выборке несколько точек — тогда усреднение и корень начинают работать по-разному (см. график ниже для примера).

MAPE ведёт себя иначе, чем остальные три метрики: она измеряет не абсолютное отклонение, а относительное (см. формулу MAPE). Это значит, что одна и та же ошибка в 2 единицы даёт разный штраф в зависимости от того, каким было истинное значение: ошибка 2 при $y = 10$ это 20%, а при $y = 1$ — уже 200%, то есть чем меньше истинное значение, тем круче кривая MAPE. Отсюда следует, что MAPE ненадёжна, когда таргет близок к нулю или равен нулю (метрика уходит в бесконечность).

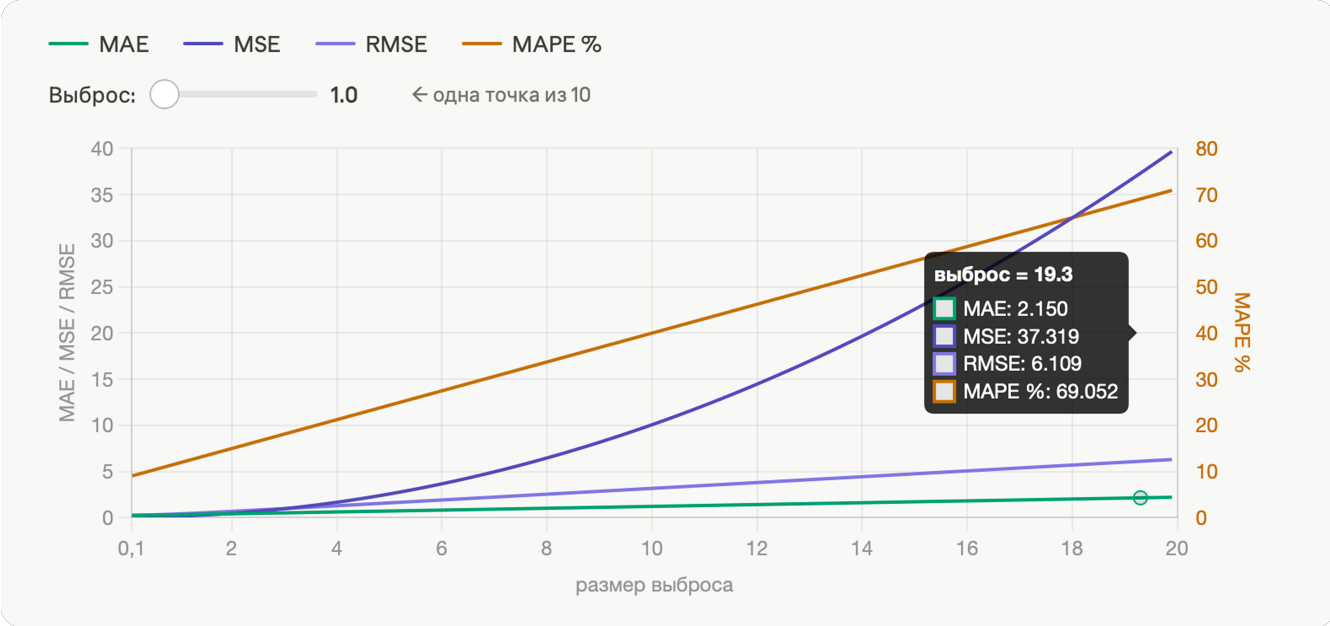


Рис. 5. Влияние выброса на поведение метрики. Сгенерировано через Claude

Рассмотрим другой пример. Представим, что мы работаем с выборкой из 10 объектов, на которых модель ошибается на 0,1–0,5 единиц таргета. На одном объекте ошибка искусственно увеличивается — это и есть выброс, размер которого регулируется слайдером.

Опустим очевидные моменты, связанные с MAE и MSE, и перейдём сразу к RMSE и MAPE.

RMSE растёт медленнее MSE — корень частично компенсирует квадратичный рост. Но она всё равно чувствительнее к выбросам, чем MAE: при выбросе 10 RMSE примерно в 3 раза выше MAE, хотя в норме они почти идентичны.

MAPE вынесена на отдельную правую ось, потому что она высчитывается в процентах и не может быть напрямую численно сравнима с остальными. Её поведение зависит от истинных значений конкретных точек, поэтому кривая может выглядеть неожиданно: выброс на точке с большим y даст меньший прирост MAPE, чем такой же выброс на точке с маленьким y.

Название	Формула	Когда использовать
MAE (mean absolute error)	$MAE = \frac{1}{n}\sum_{i=1}^{n}\lvert y_i - \hat{y}_i\rvert$	Когда важна равномерная оценка всех ошибок и устойчивость к выбросам
MSE (mean squared error)	$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$	Если нужно сильнее штрафовать крупные ошибки
RMSE (root mean squared error)	$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$	Когда важно сохранить масштаб целевой переменной
R^2 (коэффициент детерминации)	$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$	Когда нужно понять, насколько хорошо модель объясняет поведение целевой переменной
MAPE (mean absolute percentage error)	$MAPE = \frac{100\%}{n}\sum_{i=1}^{n}\left\lvert\frac{y_i - \hat{y}_i}{y_i}\right\rvert$	Когда нужно представить бизнесу результат в понятной форме или если в целевой переменной нет значений, близких к нулю

3.5. РЕГРЕССИЯ: GOOD-TO-KNOW — RMSLE, SMAPE, WAPE, MDAPE

Выше мы разобрали наиболее популярные метрики и функционалы потерь из задач регрессии. Но кроме них есть множество дополнительных метрик, которые используются реже или только в специальных задачах.

ОПРЕДЕЛЕНИЕ

RMSLE (root mean squared logarithmic error) — корень из средней квадратичной ошибки между логарифмами предсказаний и реальных значений:

$$\text{RMSLE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(log(1+\hat{y}_i) - log(1+y_i))^2}.$$

RMSLE хорошо подходит для задач, где важны относительные отклонения, модель должна «переоценивать меньше, чем недооценивать» или когда таргет сильно варьируется по порядку величины (например, в кейсах с прогнозированием продаж).

ОПРЕДЕЛЕНИЕ

sMAPE — (symmetric mean absolute percentage error) — процентная ошибка MAPE, но симметричная по отношению к истинным и предсказанным значениям:

$$\text{sMAPE} = \frac{100\%}{n}\sum_{i=1}^{n}\frac{|y_i - \hat{y}_i|}{(|y_i| + |\hat{y}_i|)/2}.$$

sMAPE работает с проблемой несимметричности обычной MAPE — одинаково «наказывает» завышения и занижения предсказаний даже при малых y_i.

ОПРЕДЕЛЕНИЕ

WAPE (weighted absolute percentage error) — отношение суммы абсолютных ошибок к сумме абсолютных истинных значений:

$$\text{WAPE} = \frac{\sum |\hat{y}_i - y_i|}{\sum |y_i|} \times 100\%.$$

WAPE — простая и интерпретируемая метрика, которая широко используется в логистике и ретейле, особенно при прогнозировании спроса и поставок.

ОПРЕДЕЛЕНИЕ

MdAPE > (median absolute percentage error) — медианная процентная ошибка — альтернатива MAPE, менее чувствительная к выбросам:

$$\text{MdAPE} = median(\left|\frac{y_i - \hat{y}_i}{y_i}\right| \times 100\%).$$

MdAPE часто применяется, когда в данных есть явные выбросы, искажающие среднее, а разработчикам нужно получить более устойчивую оценку качества модели, например, для презентации бизнесу.

3.5.1. Когда могут пригодиться good-to-know-метрики

Как уже упоминалось выше, на практике выбор метрики часто обусловлен тем, какая ошибка действительно важна бизнесу.

ПРИМЕР

Мы решаем задачу прогнозирования продаж товара в супермаркете. В этом случае обычные MAE или RMSE могут оказаться неудобными.

Ошибка в 100 единиц товара для продукта с продажами 10 000 почти незаметна, а для продукта с продажами 120 — критична.

Именно в таких случаях (задачах прогнозирования) и пригождаются метрики, работающие с **относительными**, а не абсолютными ошибками предсказаний.

- **RMSLE** полезна, когда таргет меняется на порядки величины. Возьмём тот же пример с товаром и супермаркетом — очевидно, что для дешёвого товара ошибка $10 \implies 20$ гораздо серьёзнее, чем $10000 \implies 10010$. RMSLE учитывает именно относительное отклонение, что делает её часто используемой в задачах e-commerce и demand forecasting.
- **sMAPE** широко применяется в соревнованиях и production-задачах forecasting, где нужно корректно сравнивать временные ряды разных масштабов. Например, в одном из крупных forecasting-соревнований M4 2019 sMAPE использовалась как одна из основных метрик качества.

ПРИМЕР

У автора лонгрида был проект с использованием **MdAPE**. Задача состояла в прогнозировании рекламного трафика для очередной итерации повторяющегося креатива, что превратило задачу предсказания в аналитику данных с редкими аномалиями, так как предыдущие версии креатива иногда становились вирусными. Даже один вирусный день (гигантский выброс) сильно влиял на обычную MAPE, поэтому команда использовала MdAPE, чтобы оставаться устойчивым к таким выбросам.

4. Задача классификации

От задачи регрессии перейдём к задаче классификации.

4.1. КЛАССИФИКАЦИЯ: ФУНКЦИОНАЛЫ ПОТЕРЬ

4.1.1. Log loss

ОПРЕДЕЛЕНИЕ

Log loss (logistic loss или binary cross-entropy — функция потерь, которая чаще всего используется для задач классификации.

Если нам нужно не предсказать точную метку класса, а выдать оценку (вероятность) принадлежности объекта к положительному классу, мы используем формулу:

$$y \in \{0, 1\}, p_i \in [0, 1].$$

- y — истинная метка класса.
- p_i — вероятность принадлежности объекта к положительному классу.

В этом случае функцию потерь $log\ loss$ можно записать так:

$$L_{LL}(y_i, \hat{p}_i) = -[y_i \times log(\hat{p}_i) + (1 - y_i) \times log(1 - \hat{p}_i)]$$

$$Log\ loss = -\frac{1}{N}\sum_{i=1}^{N}[y_i \times log(\hat{p}_i) + (1 - y_i) \times log(1 - \hat{p}_i)].$$

- N — общее число примеров.
- y_i — истинная метка класса (0 или 1).
- $\hat{p}_i$ — предсказанная моделью вероятность принадлежности объекта к положительному классу.

Эта формула работает как сумма двух частных случаев:

- Если $y_i = 1$, штрафуется $\log(\hat{p}_i)$: модель должна предсказывать высокую вероятность положительного класса.
- Если $y_i = 0$, штрафуется $\log(1 - \hat{p}_i)$: модель должна предсказывать низкую вероятность положительного класса.

Следовательно, log loss минимизируется, когда предсказанные вероятности близки к истинным меткам. Она не просто оценивает правильность класса, а работает с вероятностями и оптимизирует точность вероятностных оценок.

Это критично для задач, в которых важно учитывать уверенность модели (например, медицинская диагностика или кредитный скоринг).

Краткую интуицию log loss можно сформулировать так.

- Чем увереннее и точнее модель в предсказании — тем лучше.
  - Модель, предсказавшая вероятность 0,99 на правильный класс, получит очень маленький штраф

(почти 0).
    - Модель, предсказавшая вероятность 0,6 на правильный класс, получит ощутимый штраф (хотя предсказание формально правильное).
- Чем увереннее и ошибочнее — тем хуже (штраф за уверенную ошибку крайне велик).
    - Если модель предсказывает вероятность 0,99 на неправильный класс, то итоговый log loss будет огромным. Это делает log loss крайне чувствительным к уверенным ошибкам.

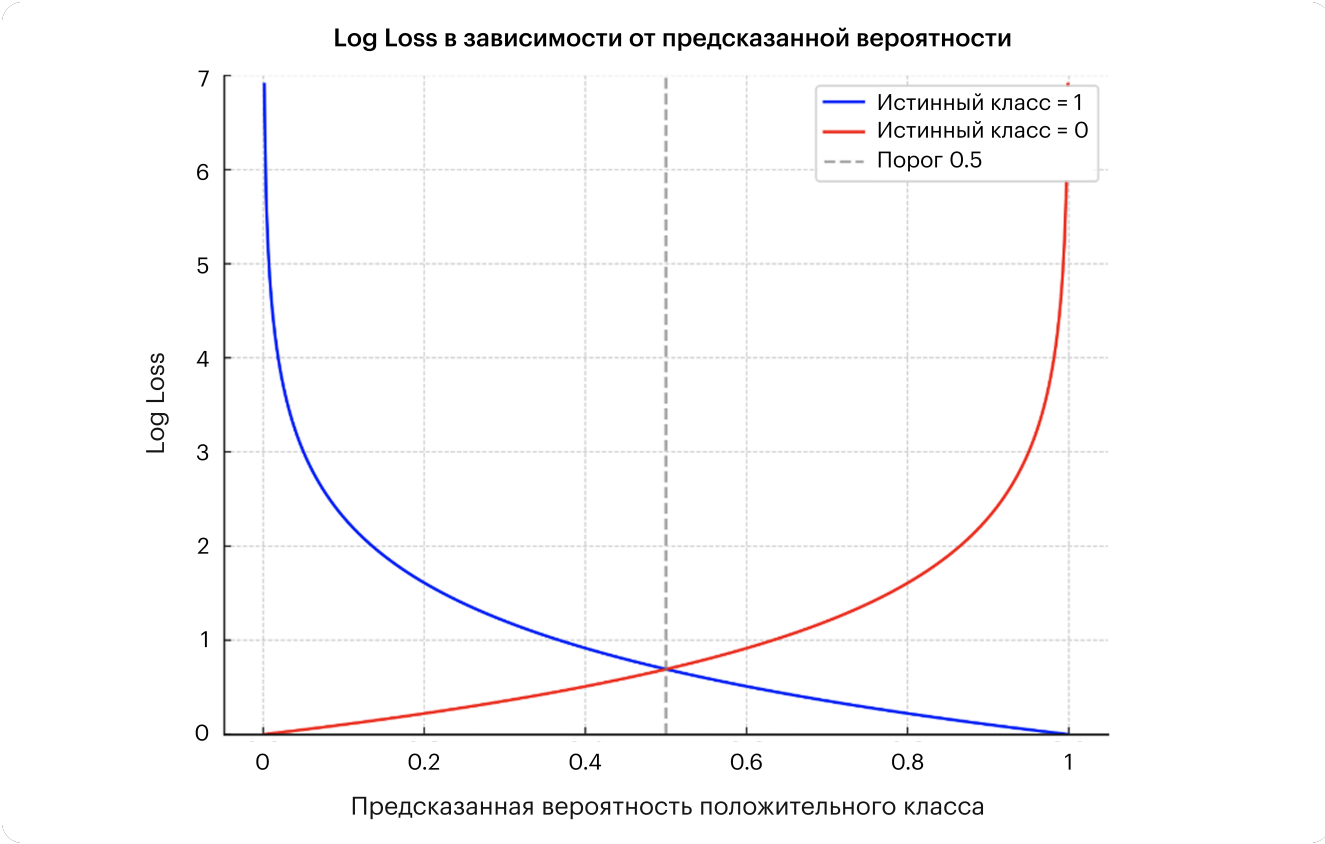


Рис. 6. Log loss в зависимости от предсказанной вероятности

Выше представлен график log loss для двух случаев.

- Истинный класс = 1 (синяя линия): потери резко возрастают при заниженных вероятностях (близко к 0) и минимальны при вероятности, стремящейся к 1.
- Истинный класс = 0 (красная линия): потери резко возрастают при завышенных вероятностях (близко к 1) и минимальны при вероятности, стремящейся к 0.

ВАЖНО

В случае log loss оптимальной константой будет доля положительного класса в выборке. Если 30% объектов имеют метку 1, то оптимально предсказывать вероятность 0,3 для всех объектов. Интуиция простая: если у нас нет никаких признаков, лучшее, что можно сделать — отразить реальное распределение классов.

ПРИМЕР

Пример расчёта

Представим задачу бинарной классификации, где модель предсказывает вероятность положительного класса.

Пусть у нас есть 3 объекта.

Объект	Истинный класс (y)	Вероятность класса 1 ($\hat{p}$)
A	1	0.9
B	1	0.6
C	0	0.8

По формуле log loss посчитаем вклад каждого объекта.

- Для объекта A:
  - модель угадала правильно с высокой степенью уверенности, а значит, по правилу log loss штраф должен быть минимальным.

$$\text{Log loss} = -1 \times (1 \times \log(0.9)) + (1 - 1) \times \log(1 - 0.9) \approx 0,105$$

- Для объекта B:
  - модель также угадала правильно, но с меньшей уверенностью, а значит, по правилу log loss штраф должен быть умеренным, но больше, чем для объекта A.

$$\text{Log loss} = -1 \times (1 \times \log(0.6)) + (1 - 1) \times \log(1 - 0.6) \approx 0,511$$

- Для объекта C:
  - модель не угадала, причём была уверена в своем ответе, а значит, по правилу Log Loss должна получить большой штраф.

$$\text{Log loss} = -1 \times (0 \times \log(0.8)) + (1 - 0) \times \log(1 - 0.8) \approx 1,609$$

Итого:

$$\text{log loss} = \frac{0.105 + 0.511 + 1.609}{3} \approx 0,742$$

4.1.2. Categorical cross-entropy loss

Log loss (binary cross-entropy) — частный случай **categorical cross-entropy loss**. Логика функции остаётся той же, меняются лишь детали формулы, поскольку вместо двух классов функция работает с C-классами.

Для задачи с C-классами:

$$L_{CE}(y_i, p_i) = -\sum_{c=1}^{C} y_{i,c} \times log(p_{i,c})$$

$$Cross{-}entropy\ loss = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} y_{i,c} \times log(p_{i,c})$$

где N — количество наблюдений (объектов); C — количество классов; $y_{i,c}$ — истинная метка (1 — если

объект i принадлежит классу c, в других случаях — 0); $p_{i,c}$ — вероятность, предсказанная моделью для класса c для объекта i.

С красивой визуализацией работы cross-entropy loss можно ознакомиться здесь.

ВАЖНО

В случае categorical cross-entropy логика оптимальной константы останется той же, что и для log loss, но для каждого класса отдельно. Оптимально будет предсказывать для каждого объекта вектор вероятностей, равный частотам классов в выборке.

Если классов три и они встречаются в соотношении 50/30/20%, то оптимально всегда предсказывать $[0.5, 0.3, 0.2]$.

4.1.3. Hinge loss

Hinge loss — это лосс алгоритма support vector machine (SVM), о котором мы подробно поговорим в рамках недели 4 этого курса. По сравнению с cross-entropy, hinge loss построен на принципиально другом подходе.

В то время как cross-entropy работает с вероятностями и штрафует за неуверенность модели в предсказании, hinge loss работает с **отступом** — расстоянием объекта от разделяющей границы между классами — и штрафует за его недостаточность.

ВАЖНО

Отступ (margin) — это расстояние объекта от разделяющей границы. В SVM граница — это не просто линия, а линия с определенным «зазором» по обе стороны.

$$L_H(y_i, \hat{y}_i) = max(0, 1 - y_i \times \hat{y}_i)$$

$$Hinge\ loss = \frac{1}{n}\sum_{i=1}^{n} max(0, 1 - y_i \times \hat{y}_i)$$

ВАЖНО

При работе алгоритма SVM стремиться сделать так, чтобы все объекты находились не просто на правильной стороне разделяющей границы, а находились там с запасом — на расстоянии не меньше 1 от границы.

Единица в формуле — это и есть требуемый отступ. Штраф обнуляется только когда $y_i \times \hat{y}_i \geq 1$, то есть когда объект находится на правильной стороне **и** на расстоянии не меньше 1 от границы. Объект может быть правильно классифицирован, но всё равно получить штраф, если он слишком близко к границе.

Число 1 является гиперпараметром — это просто масштаб, который используется по умолчанию. Реальный смысл в том, что лосс создаёт «мёртвую зону» — если объект достаточно далеко и на правильной стороне, он вообще не влияет на обучение. Модель тратит усилия только на объекты, которые либо классифицированы неверно, либо слишком близко к границе.

Представим, что перед алгоритмом стоит задача классификации One-vs-All.

В логистической регрессии модель выдаёт вероятность от 0 до 1. В SVM всё иначе, модель выдаёт просто число — любое, без ограничений, например, $+3.7$, -0.2, $+0.5$. Это число показывает, на какой

стороне от разделяющей границы находится объект и насколько далеко.

Знак числа — это предсказанный класс.

- Положительное число ⇒ класс $+1$;
- Отрицательное число ⇒ класс -1.

ВАЖНО

Hinge loss определена для меток -1 и $+1$, поэтому оптимальная константа для этой функции — это знак, а не число. Если взять производную и найти минимум по константе c:

- если положительных объектов больше, оптимально предсказывать $+1$ для всех;
- если отрицательных больше, оптимально предсказывать -1 для всех;
- если классы равны, любое значение с правильным знаком даёт одинаковый лосс.

То есть оптимальная константа для hinge loss — это **мажоритарный класс**.

Числа, выдаваемые hinge loss, не являются вероятностями. Поэтому SVM хорошо работает там, где нужно просто принять решение — спам или не спам, фрод или не фрод. Однако если бизнесу нужно знать «насколько вероятно, что это фрод», SVM с hinge loss без дополнительных преобразований не применяется.

Материалы для более углублённого погружения в тему

- Log loss (logistic loss, или cross-entropy loss)
- Гладкая функция
- Дифференцируемая функция
- Cross-entropy loss

Loss	Где использу-ется	Что оптимизи-рует	Главная идея	Оптимальная константа
Log loss (binary cross-entropy)	Бинарная классифи-кация	Вероятности классов	Штрафует уверенные ошибки	Доля положи-тельного класса в выборке
Categorical cross-entropy	Многоклассовая классифи-кация	Вероятности всех классов	Обобщение log loss для многоклас-совых случаев	Доля класса в выборке для каждого класса
Hinge loss	SVM	Margin между классами	Важно не просто угадать метку класса, а разделить классы с запасом	Мажоритарный класс

4.2. КЛАССИФИКАЦИЯ: ОСНОВНЫЕ МЕТРИКИ

Ниже представлено систематизированное описание метрик классификации в виде структурированной шпаргалки, к которой можно обратиться, чтобы освежить знания.

4.2.1. Confusion matrix

ОПРЕДЕЛЕНИЕ

Матрица ошибок (англ. confusion matrix) — базовый инструмент анализа качества бинарных классификаторов.

Она показывает:

- сколько объектов было классифицировано правильно / с ошибкой;
- какого типа эти ошибки.

Для бинарной классификации (когда показаны положительный и отрицательный классы) матрица визуализируется так.

Предсказано	Положительный	Отрицательный
Истинно положи-тельный класс (по разметке)	True Positive (TP)	False Negative (FN)
Истинно отрица-тельный класс (по разметке)	False Positive (FP)	True Negative (TN)

TP (True positive) — модель правильно предсказала положительный класс.

TN (True negative) — модель правильно предсказала отрицательный класс.

FP (False positive) — модель ошибочно предсказала положительный класс, объект принадлежит к отрицательному классу (ошибка I рода).

FN (False negative) — модель ошибочно предсказала отрицательный класс, объект принадлежит к положительному классу (ошибка II рода).

ВАЖНО

Терминология confusion matrix часто вызывает сложности. Разработчики часто путаются в словах true, false, positive и negative и их смысле в контексте confusion matrix.

Стоит запомнить,

- что true/ false отвечают за корректность предсказания (верно или не верно было сделано предсказание);
- positive/negative отвечают за предсказанный класс.

То есть false positive — модель ошиблась в предсказании, предсказав positive-класс — на самом деле класс был negative.

Для лучшего запоминания терминов иногда используют простую мнемонику.

False positive = false alarm — ложная тревога, то есть ложно предсказанный класс 1.

Эта мнемоника позволяет быстро вспомнить, что означают true, false, positive и negative.

Большинство ML-метрик, призванных оценить качество классификации, рассчитываются исходя из этих значений.

Математически confusion matrix — таблица, в которой элемент на пересечении строки и столбца ($i \times j$) обозначает количество наблюдений, фактически принадлежащих классу i, но предсказанных моделью как класс j. Следовательно, многоклассовый случай — прямое обобщение бинарного.

Если у нас есть три класса, к бинарной версии таблицы добавятся ещё одна строка и ещё один столбец. Если класса четыре, нужно добавить две строки и два столбца.

То есть для N классов матрица ошибок будет иметь размер N x N, где строки — реальные классы, а столбцы — предсказанные.

Ниже представлен наглядный пример для 3 классов (положительный, нейтральный, отрицательный — условные класс 1, 2 и 3).

	Предсказано: положительный	**Предсказано: нейтральный**	**Предсказано: отрицательный**
Истинно положительный класс	10	2	1
Истинно нейтральный класс	3	15	4
Истинно отрицательный класс	0	2	8

- На главной диагонали матрицы $[10, 15, 8]$ расположились верные ответы.
- Вне главной диагонали — ошибки, например, 3 нейтральных примера, которые были классифицированы как положительные.

Отдельно стоит отметить, что в многоклассовом случае отсутствует строго закреплённая терминология для описания сочетаний «истинное значение — предсказание». Однако принцип формирования матрицы остаётся прежним: мы подсчитываем количество соответствующих сочетаний и вносим их в соответствующую ячейку таблицы.

4.2.2. Метрики бинарной классификации

Accuracy

ОПРЕДЕЛЕНИЕ

Accuracy (accuracy score) - одна из самых интуитивно понятных метрик в задачах классификации; она показывает долю правильных предсказаний от общего числа объектов инференса и отражает общую точность работы модели.

С практической точки зрения accuracy — вероятность того, что модель даст корректный ответ на случайно выбранном объекте.

Формула accuracy выражается через элементы confusion matrix как отношение числа правильных предсказаний (TP + TN) к общему количеству наблюдений (TP + TN + FP + FN):

$$accuracy = \frac{TP + TN}{TP + TN + FP + FN}.$$

Как видно, значение accuracy лежит в диапазоне от 0 до 1, где 1 — модель, которая никогда не ошибается.

Несмотря на свою простоту, accuracy — специфичная метрика со своими проблемами и ограничениями.

- Она неустойчива при несбалансированных данных.
  Например, если 95% объектов относятся к классу 0, модель может всегда предсказывать 0 и получить accuracy = 0,95, не затрагивая класс 1.

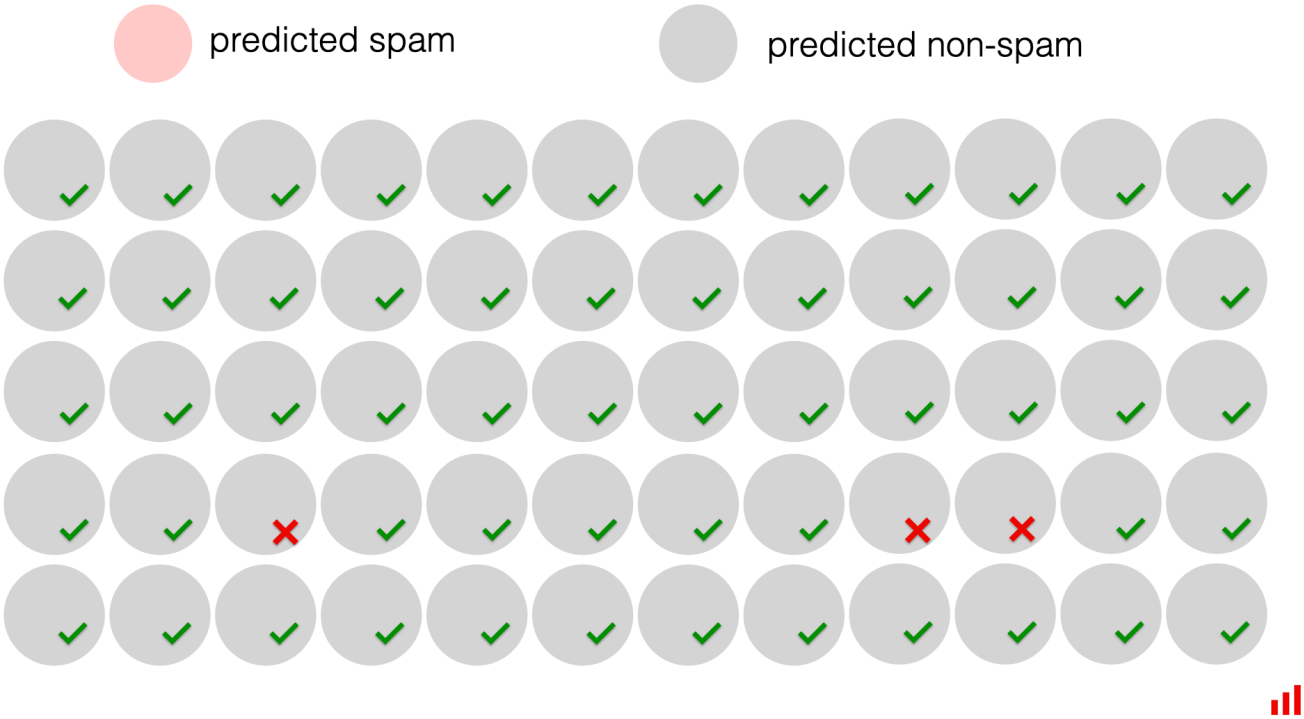


Рис. 7. Визуализация случая с несбалансированными данными

- Она не различает типы ошибок и одинаково «штрафует» за FP и FN.
  Это может быть критично в случаях, когда нужно целенаправленно оптимизировать одну из ошибок (например, только FN).
- Она не даёт полной картины происходящего и на сложных задачах может полноценно использоваться исключительно:
  - в случае сбалансированных классов,
  - комбинации с другими метриками.

Учитывая недостатки accuracy, разработчики чаще используют другие метрики, которые дают более детальное понимание качества модели. Одна из них — precision.

Precision

ОПРЕДЕЛЕНИЕ

Precision (precision score) — метрика, показывающая долю объектов, предсказанных моделью как положительные, которые действительно являются истинно положительными по разметке.

С практической точки зрения precision отвечает на вопрос: если модель говорит, что объект принадлежит к положительному классу, насколько велика вероятность, что она права?

Формула precision выражается через элементы confusion matrix как отношение числа правильных предсказаний положительного класса (TP) ко всем предсказаниям положительного класса (TP + FP):

$$precision = \frac{TP}{TP + FP}.$$

Как видно, значение precision лежит в диапазоне от 0 до 1, где 1 — модель, все положительные предсказания которой оказались верны.

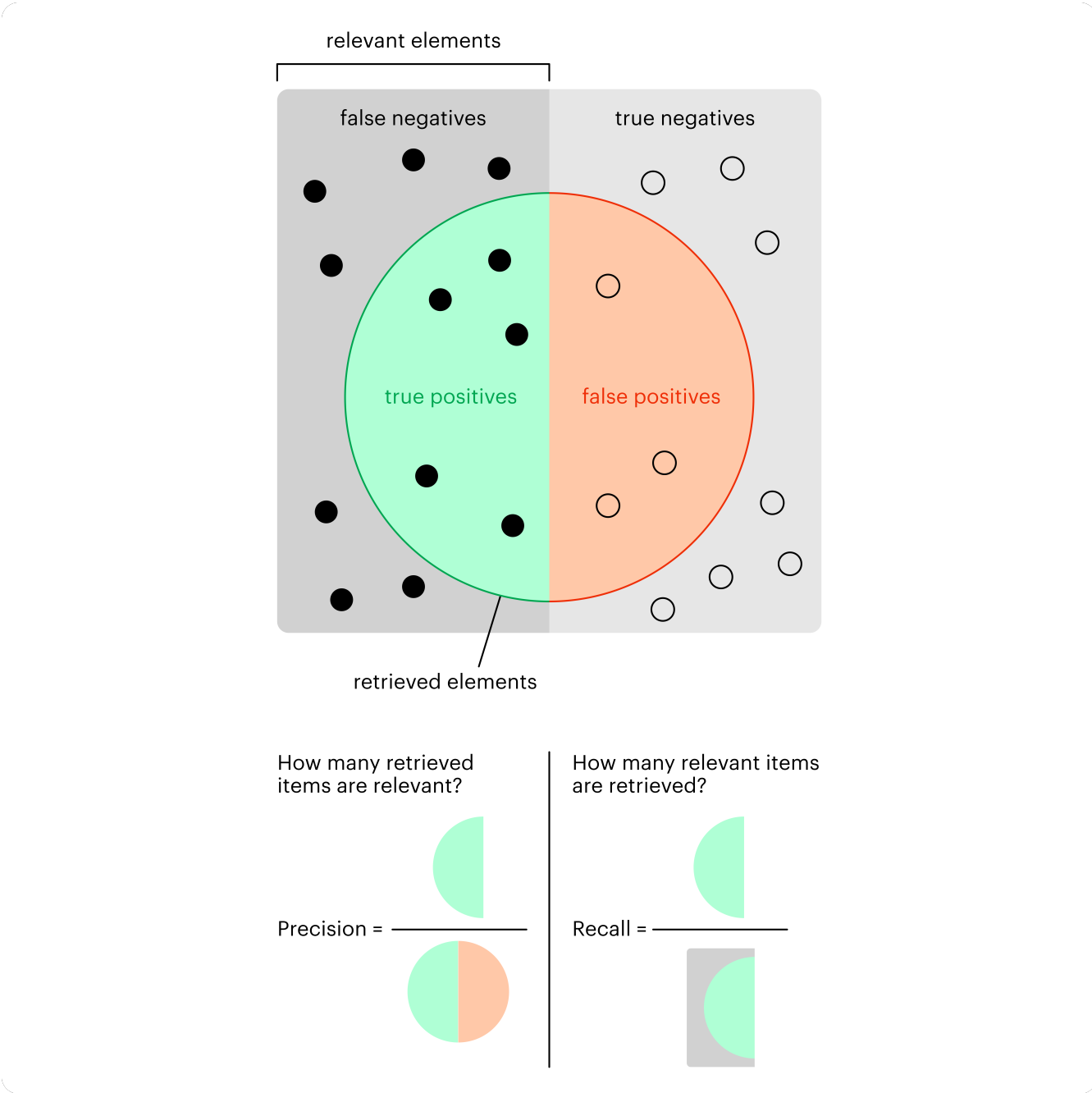


Рис. 8. Визуализация концепции precision

Precision — ключевая метрика для сфер, в которых цена ложноположительных ошибок крайне высока. Например, разработчики могут оптимизировать именно precision:

- в медицине — чтобы не ставить диагноз без оснований;
- спам-фильтрах — чтобы не терялись важные письма;
- поисковой выдаче — чтобы первые результаты были действительно полезными.

Однако, как и любая другая ML-метрика, precision имеет ряд ограничений — возможно, не таких заметных и критичных, как у accuracy.

- Она не чувствительна к пропущенным случаям.
  Метрика не показывает, сколько положительных объектов не нашла модель.

- Её можно искусственно завысить.
  Если предсказывать положительный класс очень редко, Precision будет высоким, но модель не будет качественной.
- Она неустойчива при дисбалансе классов.
  На сильно несбалансированных выборках метрика может вводить в заблуждение — precision будет высоким, даже если модель почти всегда ошибается.

Precision неразрывно связана с recall.

Recall

ОПРЕДЕЛЕНИЕ

Recall (recall score, или TPR) — это метрика, отражающая долю найденных положительных объектов.

С практической точки зрения recall отвечает на вопрос: какой процент истинно положительных объектов найдёт модель?

Формула recall выражается через элементы confusion matrix как отношение числа правильных предсказаний положительного класса (TP) ко всем истинно положительным объектам в выборке (TP + FN):

$$recall = \frac{TP}{TP + FN}.$$

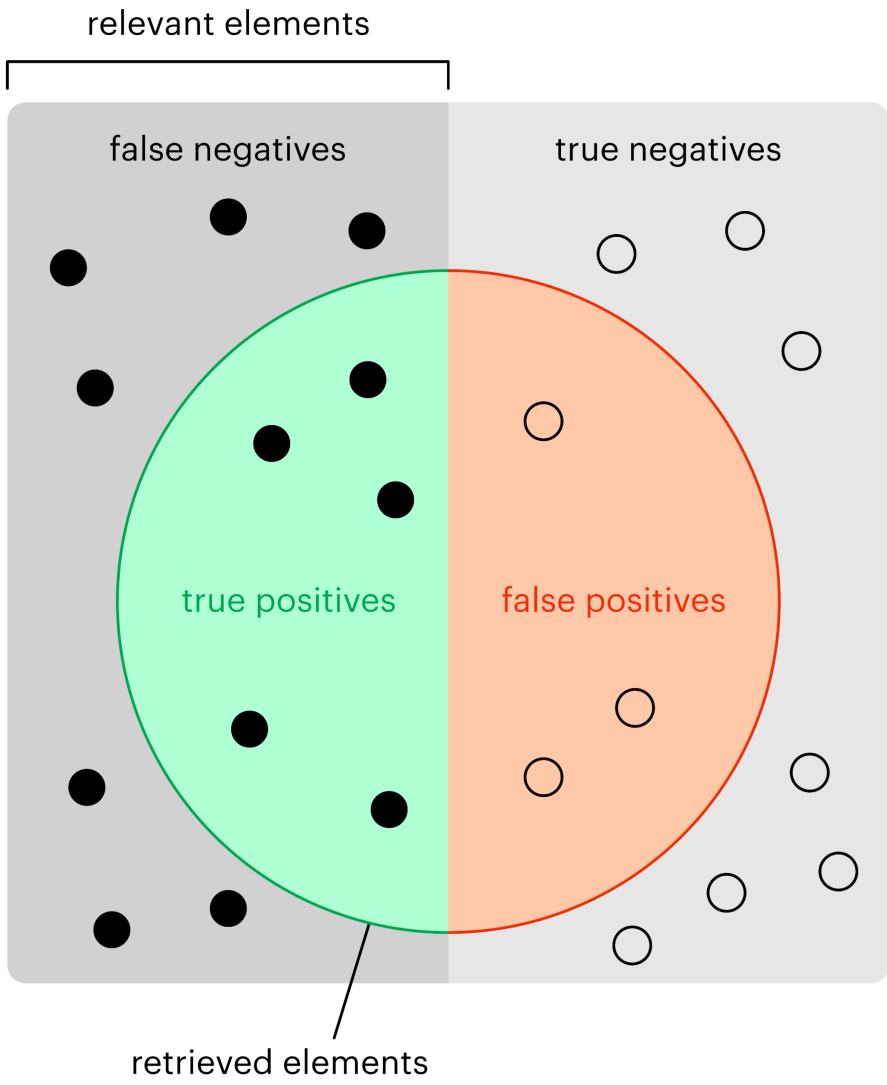
ВАЖНО

В разных контекстах recall может называться по-разному, например:

- **sensitivity** — чаще в медицине и биостатистике;
- **TPR (true positive rate)** — при построении ROC-кривых.

Нужно понимать, что все три термина (recall, sensitivity, TPR) обозначают долю правильно предсказанных положительных случаев.

Значение recall лежит в диапазоне от 0 до 1, где 1 — модель, которая нашла все истинно положительные объекты.



How many retrieved items are relevant?

Precision = $\frac{\text{true positives}}{\text{true positives} + \text{false positives}}$

Рис. 9. Визуализация концепции Recall

Recall — ключевая метрика в сферах, в которых критичен пропуск потенциально положительных случаев. Например, разработчики могут оптимизировать recall:

- в сфере безопасности — чтобы обнаружить атаки и вредоносное ПО;
- финансах и страховании — чтобы выявить случаи мошенничества и подозрительные транзакции;
- контроле качества — чтобы найти бракованные изделия на производстве.

Precision-recall tradeoff Precision и recall дополняют друг друга, позволяя всесторонне оценивать качество классификации. Однако это создаёт взаимозависимость: улучшение одной метрики часто приводит к ухудшению другой. Этот эффект и называют **precision-recall tradeoff** — компромиссом, с которым неизбежно сталкиваются при разработке модели классификации.

Заглянем чуть глубже в логику моделей классификации. Этот компромисс появляется там, где модель выдаёт оценки вероятности, а затем мы принимаем решение по определённому порогу (Threshold). Например, модель может сообщить: «Этот объект с вероятностью 0,84 относится к положительному классу». Но откуда нам знать, считать ли этот объект истинно «положительным»?

Это зависит от выбранного порогового значения (например, 0,5): если предсказанная вероятность выше 0,5, мы автоматически отнесём объект к положительному классу.

ОПРЕДЕЛЕНИЕ

Пороговое значение - гиперпараметр, который можно регулировать для достижения наилучшего результата.

Однако изменение этого порога неизбежно отразится на значениях precision и recall.

- Если мы понизим порог (например, до 0,3), то к положительному классу будет отнесено больше объектов:
  - Возрастает вероятность того, что модель захватит все положительные примеры → улучшится recall.
  - Вместе с этим она начнёт чаще ошибаться — включать в положительный класс отрицательные примеры → ухудшится precision.
- Если мы повысим порог (например, до 0,7), то итоговые предсказания будут осторожнее и с повышенной степенью уверенности:
  - Меньше ложных срабатываний → улучшится precision.
  - Но модель может пропустить важные положительные случаи → ухудшится recall.

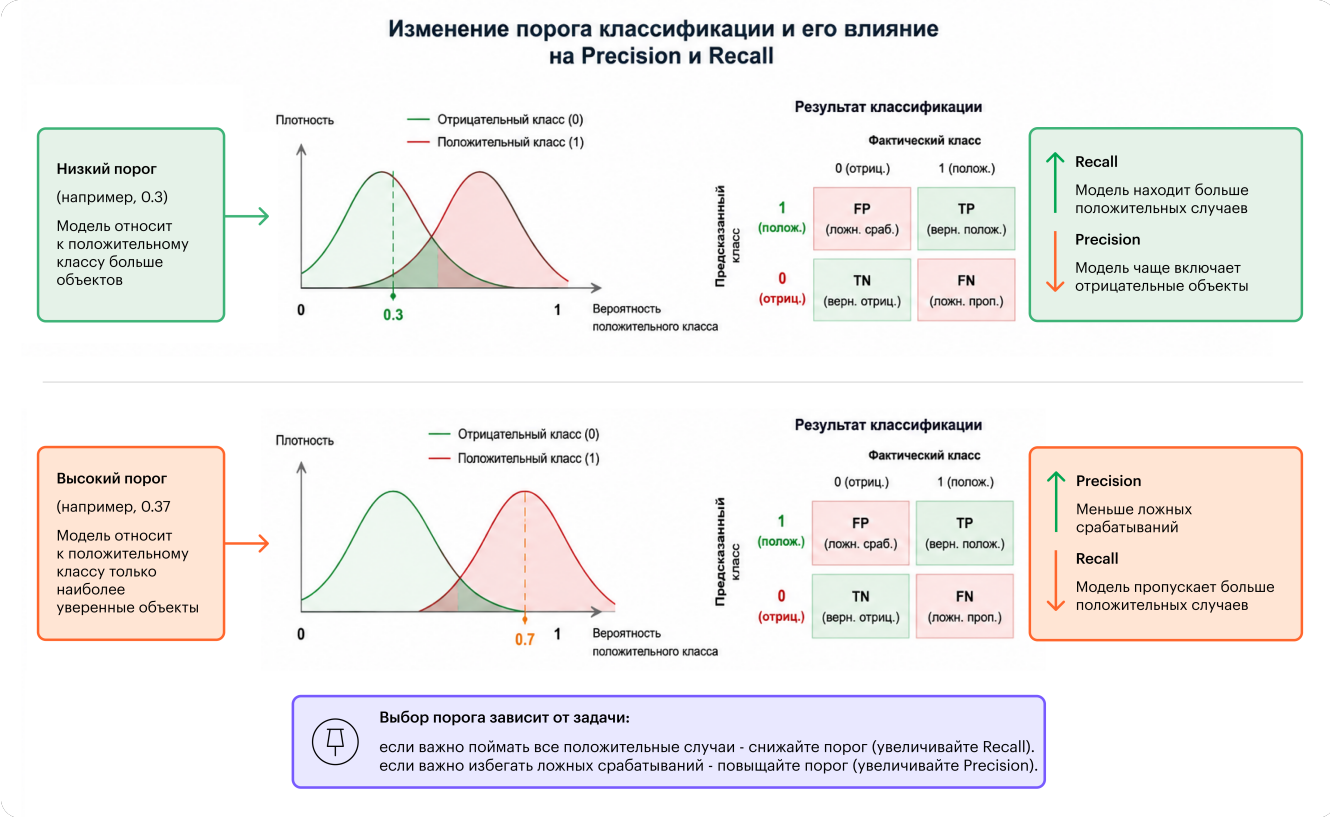


Рис. 10. Изменение порога классификации и его влияние на precision и recall

Рассмотрим пример из области медицины (медицинского скрининга):

- При низком пороге модель захватывает почти все случаи заболевания (высокий recall), но она часто путает здоровых с больными (низкий precision).
- При высоком пороге ситуация обратная: модель находит только самых «очевидных» больных (высокий precision), но упускает менее очевидные случаи (низкий recall).

Неизбежность этого компромисса вынуждает разработчиков:

- тщательнее подходить к выбору порога;
- эффективнее коммуницировать с бизнесом;
- определять, на улучшение какой метрики стоит сделать акцент.

Precision-recall curve

ОПРЕДЕЛЕНИЕ

PR-кривая (Precision-recall curve) — это способ визуализации precision-recall tradeoff, который помогает оценить компромисс между precision и recall при разных порогах классификации.

Для примера рассмотрим три порога:

$$[0.3, 0.5, 0.7].$$

Пусть модель выдала следующие вероятности положительного класса.

Вероятность positive	Вероятность negative	threshold = 0.3	threshold = 0.5	threshold = 0.7
0.4	0.6	Positive	Negative	Negative
0.6	0.4	Positive	Positive	Negative
0.55	0.45	Positive	Positive	Negative
0.1	0.9	Negative	Negative	Negative

Таким образом, чтобы построить PR-кривую необходимо сделать следующее.

1. Выбрать множество порогов.
2. Для каждого порога получить итоговые предсказания модели.
3. Рассчитать precision и recall для каждого порога.
4. Построить график.
    - Recall по оси X.
    - Precision по оси Y.

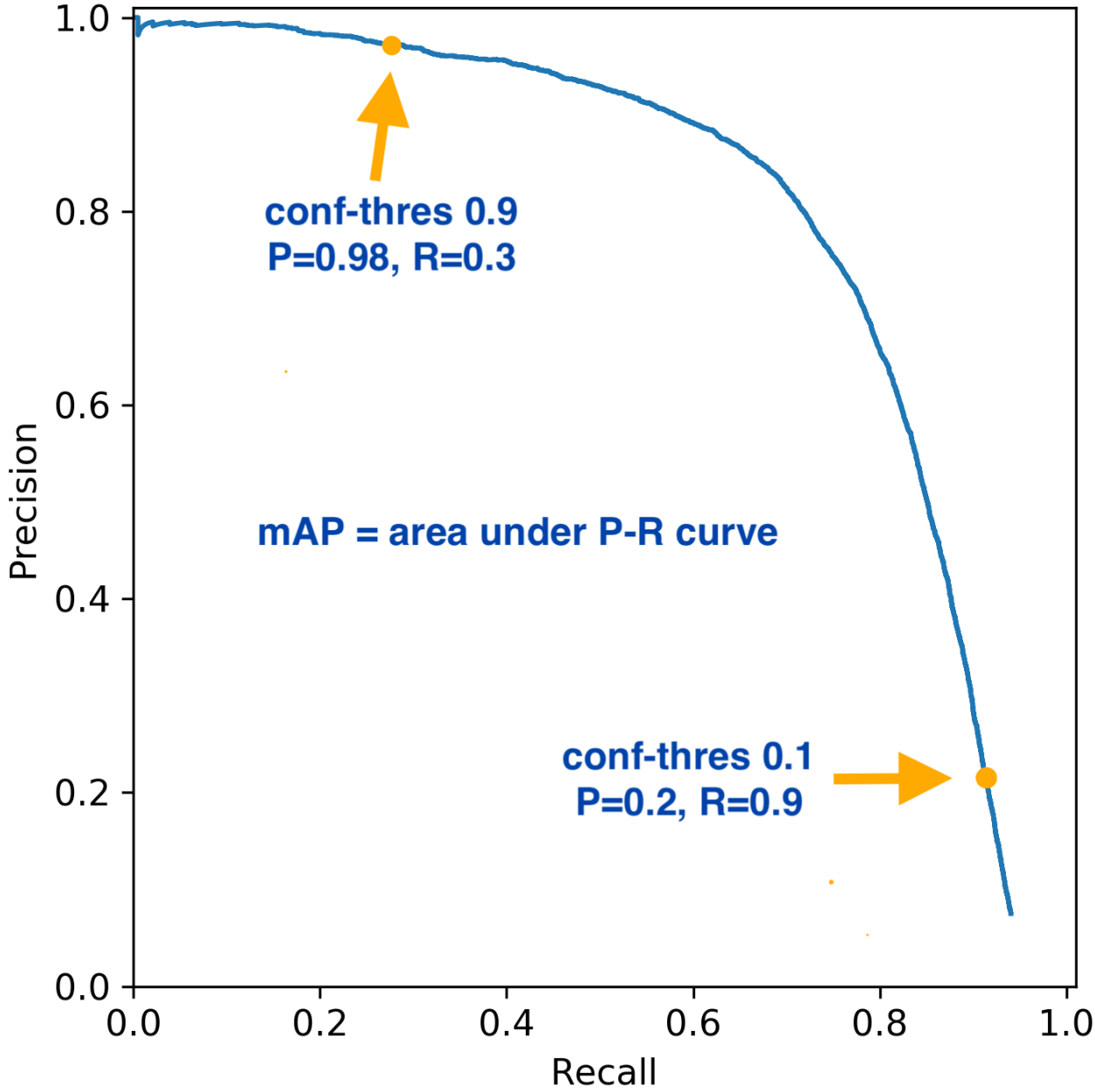


Рис. 11. PR-кривая

PR-кривая показывает, насколько хорошо модель умеет сохранять высокий precision при росте recall.

Идеальная кривая проходит через правый верхний угол графика, то есть модель имеет:

$$precision = 1,\ recall = 1.$$

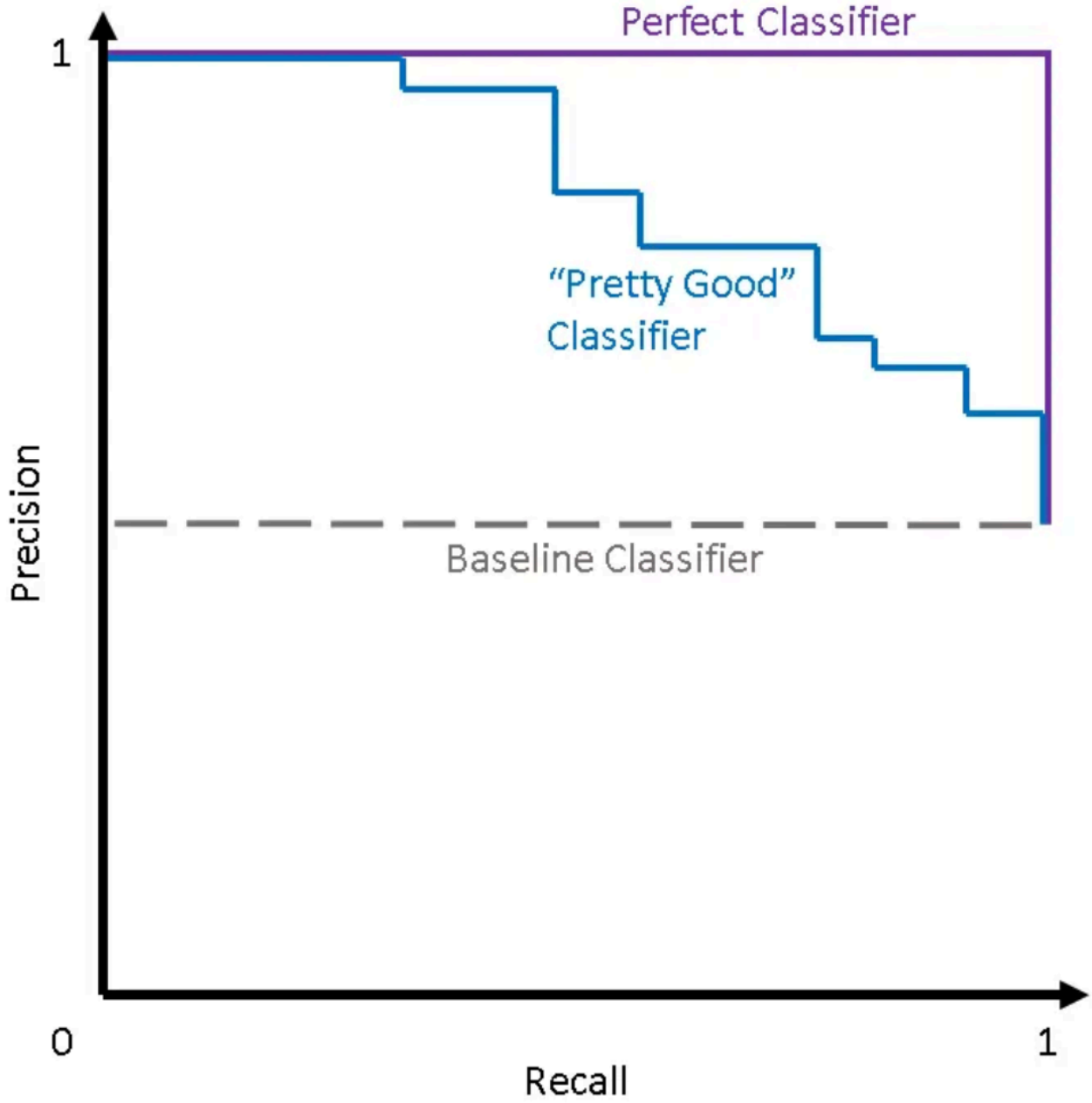


Рис. 12. Примеры гипотетических PR-кривых

F1-score и F-beta score

ОПРЕДЕЛЕНИЕ

F-beta score (F-мера) — это метрика качества классификации, которая объединяет precision и recall, позволяя регулировать их важность для задачи с помощью параметра β.

Она вычисляется по формуле:

$$F_\beta = (1 + \beta^2) \times \frac{precision \times recall}{(\beta^2 \times precision) + recall}.$$

1. При этом $\beta > 1$ смещает акцент формулы в сторону важности recall (приоритет на уменьшение коли-

чества FN-предсказаний). 2. $\beta < 1$ — в сторону важности precision (приоритет на уменьшение количества FP-предсказаний).

Это следует из формулы: поскольку в знаменателе β стоит рядом с precision, соответственно, чем он больше, тем больше влияние recall на результат, и наоборот.

ПРИМЕР

Чтобы увидеть влияние параметра β, рассмотрим случай с одинаковыми значениями метрик (precision = 0,6 и recall = 0,9).

- $\beta > 1$. Считаем F_2:

$$F_2 = (1 + 2^2) \times \frac{0{,}6 \times 0{,}9}{2^2 \times 0{,}6 + 0{,}9} = 5 \times \frac{0{,}54}{3{,}3} = 0{,}818.$$

  Модель получает достаточно высокую оценку, потому что изначальный recall, которому мы отдаём приоритет, был высок и равен 0,9.
  В реальной практике это пригодится, например, при диагностике заболеваний: важно не упустить заразных пациентов (уменьшить количество FN-ошибок). Это значит, что логично отдать предпочтение высокому recall.

- $\beta < 1$. Считаем $F_{0,5}$:

$$F_{0,5} = (1 + 0{,}5^2) \times \frac{0{,}6 \times 0{,}9}{0{,}85^2 \times 0{,}6 + 0{,}9} = 1{,}25 \times \frac{0{,}54}{1{,}05} = 0{,}64.$$

  Модель получает невысокую оценку, потому что изначальный precision, которому мы отдаём приоритет, был низок и равен 0,6.
  В реальной практике это пригодится, например, при настройке спам-фильтра. Важно, чтобы система была уверена в том, что запрос (письмо), которое она блокирует, действительно является спамом (уменьшить количество FP-ошибок). Это значит, что логично отдать предпочтение высокому precision.

- При $\beta = 1$ возникает классический **F1-score (F1-мера)** — метрика качества классификации, представляющая собой гармоническое среднее между precision и recall. Она вычисляется по формуле:

$$F_1 = 2 \times \frac{precision \times recall}{precision + recall} = \frac{2 \times TP}{2 \times TP + FP + FN}.$$

Гармоническое среднее применяется здесь неслучайно.

- Оно штрафует дисбаланс между precision и recall: если одна из величин сильно ниже другой, итоговая оценка тоже будет низкой.
- Из-за этого F1 становится чувствительна к случаям, когда модель показывает хорошие результаты по одной метрике, но проваливается по другой. Это важно в большинстве задач с несбалансированными классами.

Например, геометрическое или арифметическое среднее не так строго наказывают за дисбаланс: модель с precision = 0.99 и recall = 0.01 покажет неплохую оценку, хотя на практике она будет почти бесполезной. Это наглядно видно на примере.

Таблица 1. Сравнение средних значений для Precision и Recall

Precision	Recall	Гармоническое среднее (F1)	Арифметическое среднее	Геометрическое среднее
0.9	0.1	0.18	0.5	0.3

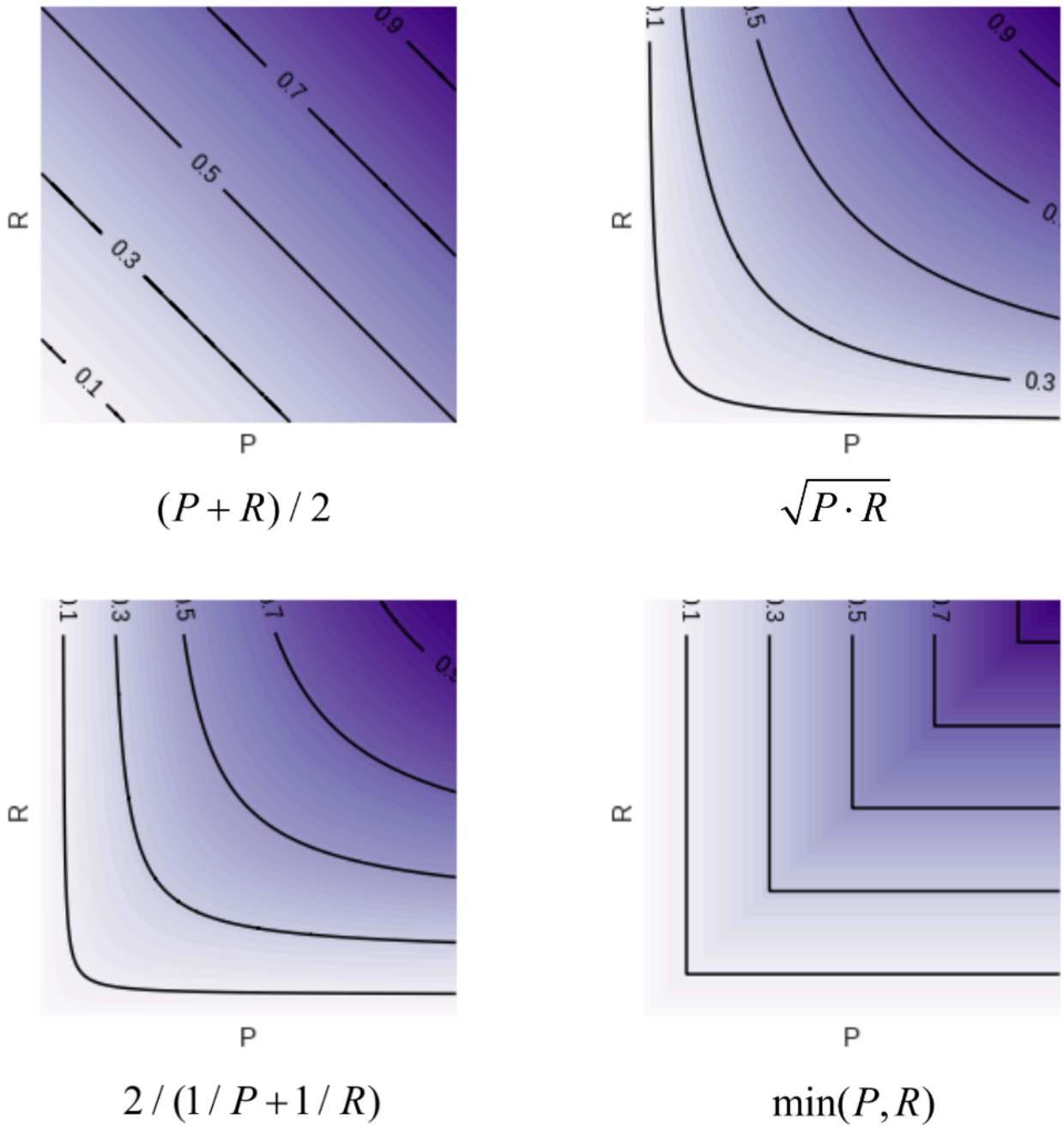


Рис. 13. Сравнение способов объединения precision и recall

На графиках выше:

- по оси X — precision;

- по оси Y — recall.

Таким образом, каждый маленький график показывает **разный способ объединить P и R в одно число**.

Цвет на графиках показывает итоговую оценку формулы: чем темнее цвет, тем она выше. Линии соединяют точки с одинаковой итоговой оценкой (то есть на линии лежат точки с разными парами P и R, которые по формуле получают одинаковую оценку). Форма линии показывает, **как метрика относится к дисбалансу**:

- **прямые линии** — метрика спокойно относится к перекосу;
- **сильно изогнутые** — требует баланса между P и R;
- **ступеньки (min)** — смотрит только на худшее значение.

То есть данная визуализация позволяет наглядно оценить, насколько конкретный способ объединения P и R «наказывает» итоговый результат за их дисбаланс.

Формула	Название	Как ведёт себя	Линии уровня на графике	Плюсы	Минусы
(P + R) / 2	Арифметическое среднее	Одинаково учитывает P и R	Прямые диагональные линии	Простая и понятная	Слабо штрафует дисбаланс
(P·R)	Геометрическое среднее	Сильнее реагирует на низкое значение	Изогнутые линии	Лучше учитывает баланс	Интерпретируется менее очевидно
2PR / (P + R)	**F1-мера (гармоническое среднее)**	Сильно снижает итог, если одна величина мала	Сильнее изогнутые линии	Хорошо балансирует precision и recall	Не отражает приоритет одной метрики над другой
min(P, R)	Минимум	Итог = худшая из двух величин	Ступенчатые области	Очень строго контролирует баланс	Часто слишком жёсткая

ВАЖНО

Именно поэтому для F1-score используют именно гармоническое среднее. С ним можно бесконечно улучшать один из показателей (например, recall), но если второй не увеличивается (precision), то качество модели всегда будет ограничено.

Нужно отметить, что F-beta score — не интуитивно понятная метрика, в особенности из-за отсутствия наглядной «физической» интерпретации (например, в отличие от accuracy). Кроме того, выбор параметра для неё не всегда очевиден. При работе с F-beta score стоит отталкиваться от следующих рассуждений:

- F-Beta Score (в частности F1-Score) крайне полезен в случаях дисбаланса классов, когда accuracy может вводить в заблуждение.
- Стоит воспринимать метрику не как абсолютный показатель качества модели, а как индикатор баланса между precision и recall.

- F1-score — наиболее часто используемая вариация F-beta score. Она по умолчанию используется в случаях, когда precision и recall одинаково важны:
  - когда нельзя упустить объекты положительного класса (recall);
  - чтобы снизить количество отрицательных объектов, ошибочно помеченных как положительные (precision);
  - если нет чёткого приоритета между этими двумя задачами.
- В случае появления явного приоритета стоит подобрать параметр beta. Однако важно помнить, что этот процесс — отражение стратегических приоритетов решаемой задачи и требует согласования с бизнес-проектом, продуктовой и аналитической командами.
- На практике на этапе экспериментов команда легко может рассчитывать несколько F-beta score с различными beta:
  - чтобы иметь возможность глубже обсуждать метрики с бизнесом;
  - продемонстрировать, как меняется оценка при смене акцентов.

TPR (true positive rate) и FPR (false positive rate) Прежде чем перейти к следующей метрике, необходимо ввести несколько дополнительных терминов, а именно: **TPR (true positive rate)** и **FPR (false positive rate)**. В задачах бинарной классификации TPR и recall полностью математически эквивалентны:

$$TPR = recall = \frac{TP}{TP + FN}.$$

Различие наблюдается лишь в контексте использования этих терминов.

- Recall используются, когда речь идёт об ML-метриках качества.
- TPR применим в контексте кривых качества (поговорим об этом ниже), порогах вероятности классификации (threshold), а также чувствительности модели.

FPR дополняет TPR, измеряя долю отрицательных примеров, которые были ошибочно классифицированы как положительные. С практической точки зрения FPR отвечает на вопрос: какова доля истинно отрицательных объектов, которые модель ошибочно предсказала как положительные?

Формула:

$$FPR = \frac{FP}{FP + TN}.$$

Как видно, значение FPR лежит в диапазоне от 0 до 1, где 0 — модель, которая не даёт ложных срабатываний (все отрицательные примеры классифицируются правильно).

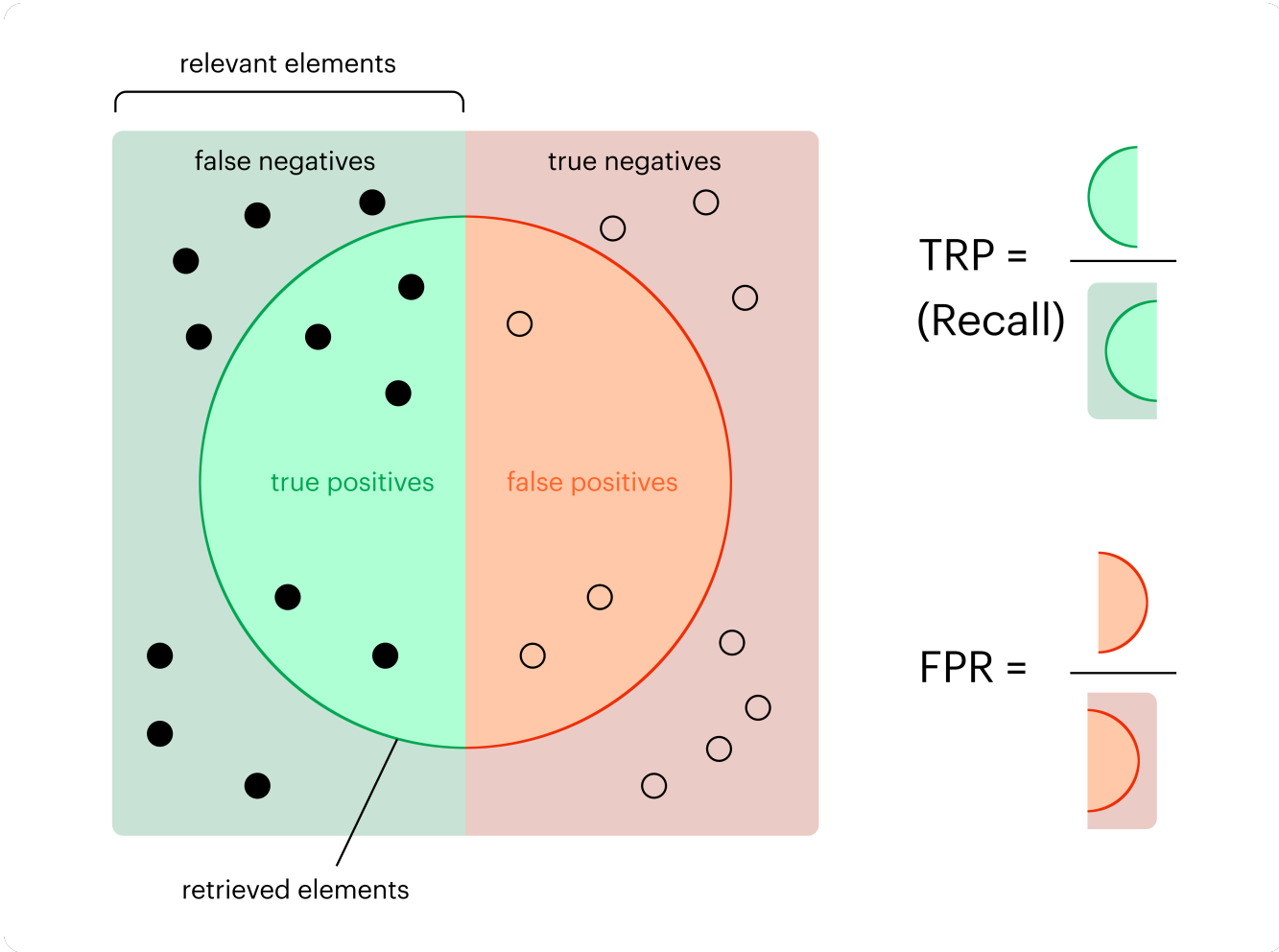


Рис. 14. Метрики true positive rate и false positive rate

ВАЖНО

Это метрика не восприимчива к значениям предсказания: важен именно порядок.

ВАЖНО

Дополнительные ссылки

- Accuracy (accuracy score)
- Precision (precision score)
- Recall (recall score, или TPR)
- F-beta score (F-мера)
- Гармоническое среднее

ROC-кривая и ROC-AUC

ОПРЕДЕЛЕНИЕ

ROC-кривая (англ. receiver operating characteristic curve) — инструмент оценки качества бинарного классификатора. В его основе — настройка предельного порога вероятности и анализ изменений TPR и FPR.

Важно отметить, что ROC-кривая оценивает эффективность работы модели через TPR и FPR при всех возможных порогах вероятности от 0 до 1. В то же время accuracy фиксирует результат на одном пороге.

Например, если модель оценивает вероятность принадлежности объекта к положительному классу выше 0,5 (порог/threshold), мы относим его к этому классу. Если вероятность 0,5 или ниже — к отрица-

тельному.

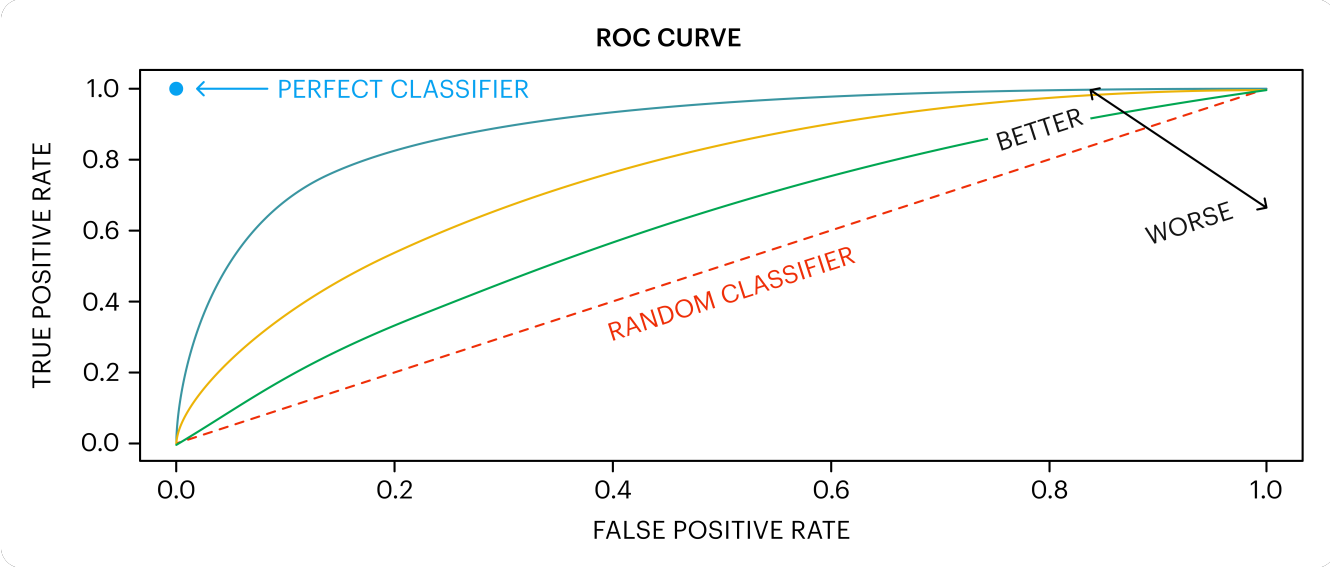


Рис. 15. Визуализация различных ROC-кривых

Алгоритм построения ROC-кривой выглядит следующим образом.

1. Модель выдаёт оценки принадлежности к положительному классу (не обязательно вероятности). Они используются для ранжирования при построении ROC-кривой. 2. Для набора порогов мы пересчитываем confusion matrix и вычисляем соответствующие значения TPR и FPR (например, 0,0, 0,01, 0,02, ..., 1,0). Чем больше этот набор:

- тем больше точек будет на кривой,
- точнее будет кривая.

3. На координатной плоскости откладываем FPR по оси X, TPR — по оси Y. 4. Соединяем полученные точки — это и есть искомая ROC-кривая.

ПРИМЕР

Алгоритм выдал оценки принадлежности объекта к классу 1, показанные в таблице №1. Упорядочив строки таблицы №1 по убыванию уверенности алгоритма, мы получим таблицу №2.

- В оптимальном случае в обеих таблицах столбец «класс» будет упорядочен: сначала идут 1, потом — 0.
- При худшем раскладе порядок будет обратный (сначала 0, потом 1).
- В случае «слепого угадывания» будет случайное распределение 0 и 1.

id	оценка	класс
1	0.5	0
2	0.1	0
3	0.2	0
4	0.6	1
4	0.2	1
5	0.3	1
7	0.0	0

Табл. 1

id	оценка	класс
4	0.6	1
1	0.5	0
6	0.3	1
3	0.2	0
5	0.2	1
7	0.1	0
2	0.0	0

Табл. 2

id	> 0.25	класс
4	1	1
1	1	0
5	1	1
3	0	0
5	0	1
7	0	0
2	0	0

Табл. 3

Рис. 16. Упорядочивание вероятностей

Чтобы нарисовать ROC-кривую, необходимо взять единичный квадрат на координатной плоскости (см. рисунок 9) и разбить его:

- на m равных частей горизонтальными линиями, где m — число 1 среди правильных меток теста (в примере $m = 3$);
- n — вертикальными, где n — число нулей (в примере $n = 4$).

В результате квадрат будет разделён сеткой $m \times n$ блоков.

Теперь просмотрим строки таблицы №2 сверху вниз и прорисуем на сетке линии, переходя из одного узла в другой. Стартуем из точки $(0, 0)$:

- если значение метки класса в просматриваемой строке = 1, то делаем шаг вверх;
- если = 0, то делаем шаг вправо.

В итоге мы попадём в точку $(1, 1)$, потому что сделали в сумме m шагов вверх и n шагов вправо.

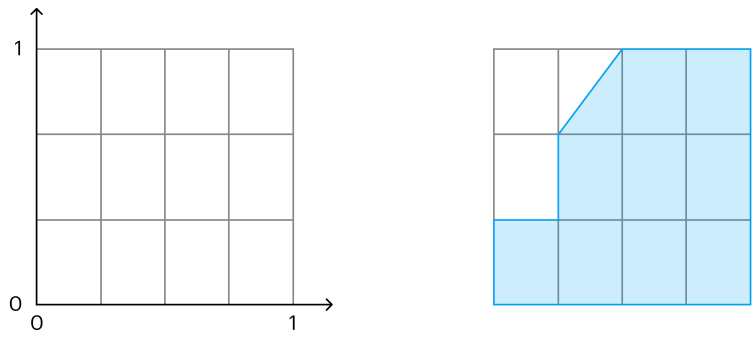


Рис. 17. Построение ROC-кривой

На рисунке выше (справа) показан путь для нашего примера — это и есть ROC-кривая.

ВАЖНО

Если у нескольких объектов значения оценок равны, то мы делаем шаг в точку, которая на a блоков выше и b блоков правее,

где a — число единиц в группе объектов с одним значением метки;

b *—* число нулей в ней. В частности, если все объекты имеют одинаковую метку, то мы сразу шагаем из точки $(0, 0)$ в точку $(1, 1)$.

Следовательно, ROC-кривая показывает всю траекторию зависимости качества модели от используемого порога, что позволяет оценить:

1) устойчивость решения к выбору порога;

2) общее качество ранжирования примеров в зависимости от этого порога.

В лучшем случае при одном из порогов кривая пересекает точку $(0, 1)$, где $FPR = 0$, а $TPR = 1$. Если этого не происходит, площадь под ROC-кривой **(area under curve, или AUC)** станет агрегированной метрикой, которая отражает общее качество модели.

ВАЖНО

Интуитивно AUC отражает вероятность того, что модель верно ранжирует случайно выбранные положительный и отрицательный примеры, присвоив первому более высокую вероятность. Интерактивный график поможет понять поведение ROC-кривой.

AUC можно интерпретировать следующим образом.

- $AUC = 1{,}0$ — идеальный классификатор.
- $AUC = 0{,}5$ — модель работает на уровне случайного угадывания.
- $AUC < 0{,}5$ — модель систематически путает классы (если инвертировать выход, то может получиться неплохое решение).

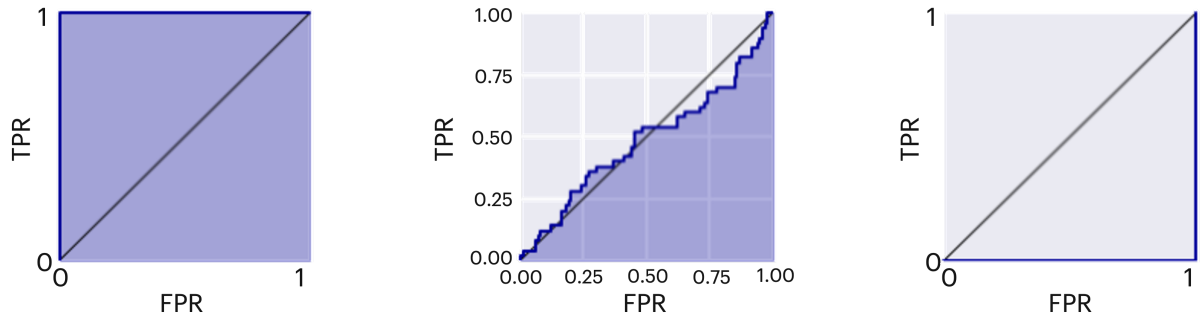


Рис. 18. Примеры различных ROC-кривых

ПРИМЕР

Возьмём кривую, которую мы строили ранее. Сетка разбила квадрат на 3×4 блоков. Ровно столько же пар вида (объект класса 1, объект класса 0) составлено из объектов тестовой выборки.

- Каждый закрашенный блок на рисунке соответствует паре, для которой алгоритм правильно предсказал порядок (объект класса 1 получил оценку выше, чем объект класса 0).
- Каждый незакрашенный блок — паре, на которой ошибся.

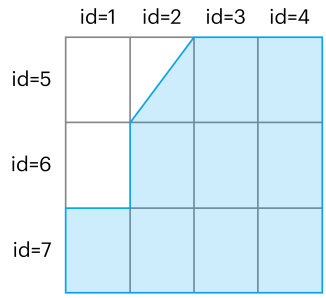


Рис. 19. Финальный результат

Как мы видим, AUC-ROC равен доле пар объектов вида, которые алгоритм упорядочил верно — первый объект идёт в упорядоченном списке раньше. Формулой это можно записать так.

$$\frac{\sum_{i=1}^{q}\sum_{j=1}^{q} I[y_i < y_j] I[a_i < a_j]}{\sum_{i=1}^{q}\sum_{j=1}^{q} I[y_i < y_j]}$$

$$I[a_i < a_j] = \begin{cases} 0, & a_i > a_j, \\ 0.5, & a_i = a_j, \\ 1, & a_i < a_j, \end{cases}$$

$$I[y_i < y_j] = \begin{cases} 0, & y_i \geq y_j, \\ 1, & y_i < y_j, \end{cases}$$

В данной формуле:

- a_i — ответ алгоритма на i-м объекте;
- y_i — его метка (класс);
- q — число объектов в тесте.

Ниже на рисунке представлены поэтапные расчёты и построение ROC-кривой для таблицы 1 с рисунка 8.

Шаг 0

id	score	>= 0.7	class
4	0.6	0	1
1	0.5	0	0
6	0.3	0	1
5	0.2	0	1
3	0.2	0	0
2	0.1	0	0
7	0.0	0	0

TP=0, FP=0 TPR=0/3=0.00 FPR=0/4=0.00
ROC point = (FPR, TPR) = (0.00, 0.00)

Шаг 1

id	score	>= 0.6	class
4	0.6	1	1
1	0.5	0	0
6	0.3	0	1
5	0.2	0	1
3	0.2	0	0
2	0.1	0	0
7	0.0	0	0

TP=1, FP=0 TPR=1/3=0.33 FPR=0/4=0.00
ROC point = (FPR, TPR) = (0.00, 0.33)

Шаг 2

id	score	>= 0.5	class
4	0.6	1	1
1	0.5	1	0
6	0.3	0	1
5	0.2	0	1
3	0.2	0	0
2	0.1	0	0
7	0.0	0	0

TP=1, FP=1 TPR=1/3=0.33 FPR=1/4=0.25
ROC point = (FPR, TPR) = (0.25, 0.33)

Шаг 3

id	score	>= 0.3	class
4	0.6	1	1
1	0.5	1	0
6	0.3	1	1
5	0.2	0	1
3	0.2	0	0
2	0.1	0	0
7	0.0	0	0

TP=2, FP=1 TPR=2/3=0.67 FPR=1/4=0.25
ROC point = (FPR, TPR) = (0.25, 0.67)

Шаг 4

id	score	>= 0.2	class
4	0.6	1	1
1	0.5	1	0
6	0.3	1	1
5	0.2	1	1
3	0.2	1	0
2	0.1	0	0
7	0.0	0	0

TP=3, FP=2 TPR=3/3=1.00 FPR=2/4=0.50
ROC point = (FPR, TPR) = (0.50, 1.00)

Шаг 5

id	score	>= 0.1	class
4	0.6	1	1
1	0.5	1	0
6	0.3	1	1
5	0.2	1	1
3	0.2	1	0
2	0.1	1	0
7	0.0	0	0

TP=3, FP=3 TPR=3/3=1.00 FPR=3/4=0.75
ROC point = (FPR, TPR) = (0.75, 1.00)

Шаг 6

id	score	>= 0.0	class
4	0.6	1	1
1	0.5	1	0
6	0.3	1	1
5	0.2	1	1
3	0.2	1	0
2	0.1	1	0
7	0.0	1	0

TP=3, FP=4 TPR=3/3=1.00 FPR=4/4=1.00
ROC point = (FPR, TPR) = (1.00, 1.00)

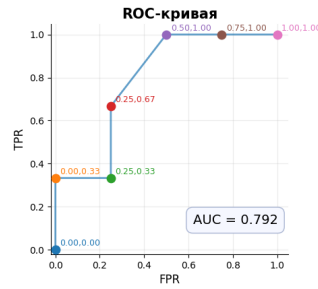


Рис. 20. Пошаговые расчеты построения ROC-кривой

ВАЖНО

ROC-AUC не чувствителен к точным значениям вероятностей, предсказываемых моделью. Имеет значение лишь то, насколько правильно она упорядочивает объекты: чем выше вероятность у положительного примера по сравнению с отрицательным, тем лучше. Поэтому даже плохо откалиброванные вероятности могут давать высокое значение ROC-AUC, если порядок ранжирования сохраняется.

Также важно отметить, что даже при одинаковом AUC ROC-кривые разных моделей могут различаться по форме. Например, одна модель может иметь хороший TPR при низком FPR в области низких порогов, а другая — наоборот, показывать стабильность только при высоких порогах. Такая разница может быть критичной в задачах, в которых важно контролировать либо FP-, либо FN-результаты.

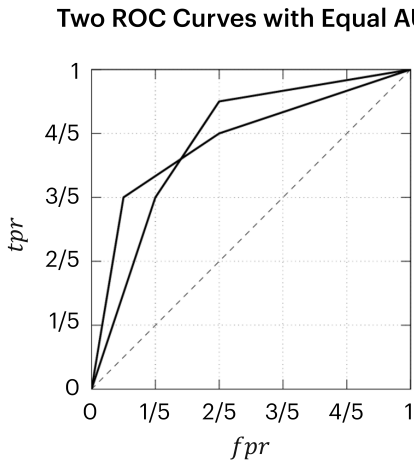


Рис. 21. Разные ROC-кривые, но с одинаковым AUC

Поэтому при анализе ROC-кривых рекомендуется анализировать форму кривой в тех ключевых зонах FPR/TPR, которые релевантны конкретной задаче и бизнес-ценности, а не опираться только на агрегированный показатель AUC.

Дополнительные ссылки

- ROC-кривая и ROC-AUC
- ROC-кривая

4.3. БИНАРНАЯ КЛАССИФИКАЦИЯ: ОСНОВНЫЕ МЕТРИКИ (РЕКАП)

id	Истинный класс	Предсказанная вероятность P(1)	Предсказанный класс (порог = 0.5)	
1	1	0.96	1	TP
2	1	0.91	1	
3	1	0.84	1	
4	1	0.72	1	
5	1	0.41	0	FN
6	1	0.35	0	
7	0	0.78	1	FP
8	0	0.44	0	TN
9	0	0.21	0	
10	0	0.08	0	

1. Accuracy (доля верных предсказаний)

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{4 + 3}{4 + 3 + 1 + 2} = \frac{7}{10} = 0.7$$

2. Precision (точность)

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{4}{4 + 1} = \frac{4}{5} = 0.8$$

3. Recall (полнота)

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{4}{4 + 2} = \frac{4}{6} = 0.67$$

4. F1 (среднее гармоническое между Precision и Recall)

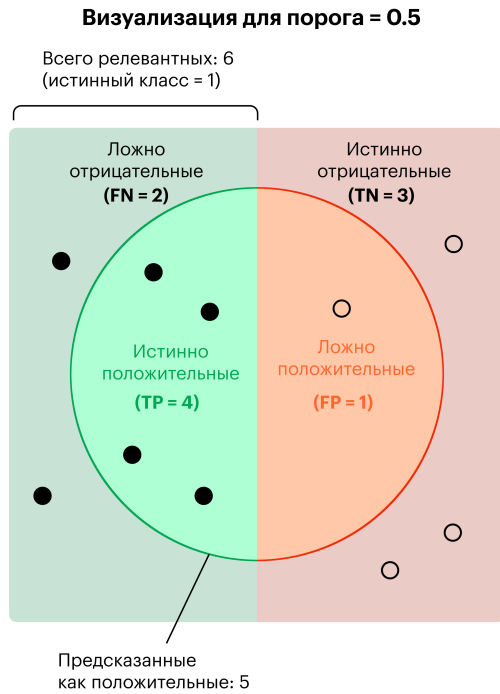
$$F1 = \frac{2 * TP}{2 * TP + FP + FN} = \frac{2 * 4}{2 * 4 + 1 + 2} = \frac{8}{11} = 0.73$$

5. True Positive Rate (TPR, Recall)

$$TPR = \frac{TP}{TP + FN} = \frac{4}{4 + 2} = \frac{4}{6} = 0.67$$

6. False Positive Rate (FPR)

$$FPR = \frac{FP}{FP + TN} = \frac{1}{1 + 3} = \frac{1}{4} = 0.25$$



Матрица ошибок

	Предсказанный 1	Предсказанный 0
Истинный 1	TP = 4	FN = 2
Истинный 0	FP = 1	TN = 3

Рис. 22. Метрики бинарной классификации

5. Классификация: обобщение на многоклассовые случаи

Выше мы разобрали метрики для бинарного случая постановки задачи классификации. Когда речь заходит о многоклассовых или многометочных случаях, метрики меняются.

5.1. МИКРО-, МАКРО-, ВЗВЕШЕННЫЕ УСРЕДНЕНИЯ

При работе над задачей многоклассовой классификации (число классов > 2) можно:

1) воспользоваться one-vs-all-подходом (считать один класс позитивным, а все остальные — негативными);

2) рассчитать precision, recall и F-beta score для каждого класса отдельно.

Но для получения сводной оценки модели по всем классам результат такого подсчёта необходимо агрегировать в единое итоговое значение. Для этого существует несколько видов усреднения.

Macro-averaging (макроусреднение). Оно предлагает:

1) рассчитать выбранную метрику (например, precision, recall или F1) отдельно для каждого класса;

2) усреднить их результат.

Например, для многоклассовой precision формула с макроусреднением выглядит так:

$$Precision_{macro} = \frac{1}{K}\sum_{i=1}^{K} Precision,$$

где K — количество классов.

Для такого формата усреднения все классы будут иметь равный вес вне зависимости от количества объектов в каждом классе. Это наиболее подходящий вариант для задач, в которых важно уделить одинаковое внимание всем категориям, включая редкие (например, для задач антифрода или медицинской диагностики).

Micro-averaging (микроусреднение). Оно предлагает:

1) построить one-vs-all confusion matrix для каждого класса;

2) вычислить суммарные TP, FP и FN по всем классам;

3) рассчитать финальное значение выбранной метрики на основе полученных результатов.

Стоит обратить внимание на то, что в этом случае нет расчёта метрик по каждому классу отдельно — всё агрегируется сразу.

Например, для многоклассовой precision формула с микроусреднением выглядит так:

$$Precision_{micro} = \frac{\sum_{i=1}^{K} TP_i}{\sum_{i=1}^{K} (TP_i + FP_i)}.$$

Для такого формата усреднения все отдельные предсказания будут иметь одинаковый вес вне зависимости от класса. Это особенно хорошо работает в задачах, в которых:

- важно учитывать общий баланс правильно/неправильно классифицированных объектов;
- модель в первую очередь должна быть точной в абсолютных числах по всем наблюдениям.

Weighted-averaging (взвешенное усреднение). Оно предлагает:

1) рассчитать выбранную метрику (например, precision, recall или F1) отдельно для каждого класса;

2) усреднить их с учётом доли каждого класса (вес пропорционален количеству примеров класса).

В итоге классы с большим числом объектов сильнее повлияют на итоговый результат.

Например, для многоклассовой precision формула с взвешенным усреднением выглядит так:

$$Precision_{weighted} = \frac{\sum_{i=1}^{K} Precision_i \times n_i}{\sum_{i=1}^{K} n_i},$$

где n_i — количество объектов класса i.

Такой формат усреднения — компромисс между микро- и макрослучаями: классы с большим числом примеров оказывают большее влияние, но малочисленные тоже учитываются. Взвешенное усреднение будет полезно в случаях, когда нужно учитывать долю классов в выборке, не забывая следить и за качеством предсказаний на редких классах.

Вид усреднения	**Как считается**	**Что особенно важно**	**Когда использовать**
Macro	Среднее по классам	Класс	Когда важно, чтобы предсказания на всех классах (в том числе редких) были точными
Micro	Сумма по наблюдениям	Предсказание	Когда наблюдается сильный дисбаланс классов и важна общая точность модели вне зависимости от класса
Weighted	Среднее по классам с весами	Класс с учётом веса	Когда нужен компромисс между micro и macro, а также в production-случаях

ПРИМЕР

Представим задачу классификации обращений в службу поддержки:

- A — обычные вопросы;
- B — проблемы с оплатой;
- C — критические инциденты.

Распределение классов

Класс	Количество объектов
A	900
B	90
C	10

Допустим, модель демонстрирует такой F1-score по классам.

Класс	F1-score
A	0.98
B	0.70
C	0.10

- **Макроусреднение:**

$$F_{1_{macro}} = \frac{0.98 + 0.7 + 0.1}{3} \approx 0.593.$$

  - Хотя модель идеально работает на массовом классе A, она почти полностью провалила критический класс C, и **макроусреднение** сильно штрафует модель за это.
  - Такое усреднение применяется, когда редких объектов мало, но пропускать их никак нельзя, например, fraud detection, медтех, модерация запрещенного контента.
- **Микроусреднение:**

$$F_{1_{micro}} \approx 0.95$$

  - Не будем вдаваться в математику, интуитивно по логике **микроусреднения** класс A огромный, модель работает на нём почти идеально, а значит и общий результат будет очень высоким.
  - Ошибки на маленьком классе C почти не повлияют на итоговую оценку.
  - Такое усреднение применяется, когда важна общая точность системы на потоке объектов, например, рекомендательные системы, поиск, intellectual document processing.
- **Взвешенное усреднение:**

$$F_{1_{weighted}} = \frac{0.98 \times 900 + 0.7 \times 90 + 0.1 \times 10}{1000} \approx 0.946$$

  - Получившийся итог почти такой же высокий, как у micro, но провал класса C всё-таки внесёт свою лепту и немного ухудшит результат.
  - Такое усреднение применяется, когда важно хорошо работать на массовых сценариях, но полностью игнорировать редкие классы тоже нельзя,

например, в e-commerce, ad ranking и customer support.

5.2. HAMMING SCORE

ВАЖНО

Логика, описанная выше для многоклассовой классификации, применима и в многометочном случае — так, используя Hamming score, вместо классов будем взаимодействовать с возможными метками.

ПРИМЕР

Hamming score — это ML-метрика для многометочных задач, которая показывает, какая доля всех меток (положительных и отрицательных) была предсказана верно.

Формула:

$$Hamming\ score = \frac{1}{n_{samples} \times n_{labels}} \sum_{i=1}^{n_{samples}} \sum_{j=1}^{n_{labels}} 1(y_{i,j} = \hat{y}_{i,j}),$$

где $y_{i,j}$ — истинное значение метки j для объекта i;

$\hat{y}_{i,j}$ — предсказанная моделью метка;

$1(.)$ — индикатор (1 — если условие выполняется, в других случаях — 0).

ВАЖНО

В многометочных задачах метки каждого объекта могут быть либо положительными (метка присутствует у определённого объекта), либо отрицательными (метка отсутствует).

Учёт всех меток — ключевое отличие Hamming score от метрик, рассмотренных выше. Эта разница существенно влияет на интерпретацию, делая её более подходящей для многометочных задач.

ПРИМЕР

Есть три класса. Истинная метка объекта включает класс 1 и класс 2, но классификатор предсказал для этого объекта только класс 1. Как в этой ситуации оценить предсказание accuracy и Hamming score?

- **Accuracy.** Предсказание считается неправильным, потому что набор предсказанных классов не совпадает с истинным.

$$Accuracy = 0.$$

- **Hamming score.** Истинную метку разбивают на две части: класс 1 и класс 2. В данном случае модель:
  - правильно предсказала одну часть;
  - ошиблась в другой;
  - верно не отнесла объект к третьему классу;

$$Hamming\ score = \frac{2}{3}.$$

> Мы видим, что Hamming score более сбалансирован и строже оценивает общую точность по всем меткам, что идеально подходит для многометочных случаев.

5.3. ROC-КРИВАЯ В МНОГОКЛАССОВОМ СЛУЧАЕ

Классическая ROC-кривая определена для бинарной классификации. Но в многоклассовом случае очевидного «положительного» класса просто нет. Поэтому при построении ROC-кривой для многоклассовой задачи необходимо свести задачу к бинарной. На практике используют следующие подходы.

1. **One-vs-Rest (OvR)** Для каждого класса строится свой ROC: «этот класс» против «все остальные». То есть в случае классификации на 3 класса будут построены 3 ROC-кривые.
2. **One-vs-One (OvO)** Для каждой пары классов строится своя ROC-кривая.
3. **Micro-average ROC**
4. **Macro-average ROC**

5.3.1. Micro-average ROC

Сперва для каждого класса считается своя One-vs-All задача. Затем Все One-vs-All предсказания складываются в один большой бинарный массив, к которому уже применяется стандартный алгоритм threshold, на основе которого строится единая ROC-кривая.

То есть micro-average ROC усредняет не метрики отдельных классов, а смотрит на сами объекты и ошибки классификации. Это позволяет большим классам сильнее влиять на итоговую ROC-кривую, которая в целом демонстрирует общее качество модели и то, насколько хорошо модель работает в среднем по всем объектам.

5.3.2. Macro-average ROC

Для каждого класса отдельно строится своя One-vs-All ROC-кривая, например:

- ROC для A vs All;
- ROC для B vs All;
- ROC для C vs All.

У каждой из этих кривых:

- свои вероятности;
- свои thresholds;
- свои точки TPR/FPR.

Затем macro-average ROC-кривые (их TPR/FPR точки) усредняют на одинаковых threshold, за счёт чего все классы получают одинаковый вес вне зависимости от размера изначального размера.

5.4. ВЫБОР ML-МЕТРИКИ И ФУНКЦИОНАЛА ПОТЕРЬ

5.4.1. Примеры: как может выглядеть логика подбора ML-метрики и loss function

Классификация

- Если классы сбалансированы, можно использовать ML-метрику accuracy (долю правильно предсказанных меток). Она не дифференцируема и как loss использоваться не может, зато часто используется как метрика в задачах классификации.
- Если один класс встречается гораздо реже (например, «фрод» или «плохой клиент»), приоритетом становится recall по минорному классу — доля корректно найденных редких объектов.
- В бинарной классификации распространённым loss является log loss (или binary cross-entropy). Log loss штрафует модель пропорционально уверенности в ошибочном предсказании: сказал «вероятность 0,99», а на самом деле класс другой — штраф огромный. Это делает log loss мощным инструментом с отличным «обучающим сигналом».

Регрессия

- **MSE** (среднеквадратичная ошибка) — чувствительна к выбросам, так как ошибки возводятся в квадрат. При этом ее можно использовать и как **ML-метрику** и как **loss**, ведь MSE дифференцируема, выпукла, градиент считается легко.
- **RMSE** — это просто корень из MSE. Как loss RMSE почти не используют (градиент сложнее), зато как метрику его можно встретить часто, так как RMSE выражается в тех же единицах, что и целевая переменная, и проще интерпретируется.
- MAE (средняя абсолютная ошибка) — менее чувствительна к экстремальным значениям, но сложнее оптимизируется.
- Если бизнес-задача связана с минимизацией крупных ошибок, выбирают можно использовать MSE как loss и MSE/RMSE как метрику. Если важна стабильность — будет лучше попробовать MAE и как **loss** и как **метрику**.

Выбирать ML-метрику и доss нужно максимально осознанно.

Правильный выбор ML-метрик и доss — один из факторов, определяющий успех решения. Выбирать их стоит максимально осознанно. Для этого нужны насмотренность и особый мыслительный подход: нужно определить, что действительно важно в задаче, а чем можно пренебречь.

5.4.2. Когда метрика и loss совпадают

Как мы уже обозначили выше, иногда одна и та же функция может выступать и как лосс, и как метрика — и это нормально. Например:

- **MSE** вполне можно и оптимизировать при обучении, и использовать как метрику оценки;
- **log loss** также используют и как лосс при обучении логистической регрессии, и как метрику при сравнении моделей.

Возможность некоторых функций выступать и как loss, и как метрика — не ошибка, а просто частный случай и удачное стечение обстоятельств. Главное — понимать, что далеко не все функции можно использовать схожим образом, а также выбирать loss и метрику с умом, а не просто использовать одну функцию на двоих, потому что так удобно или по привычке.

6. Если кратко

- Курс про фундамент классического ML. Без нейросетей, с упором на базовые алгоритмы, обработку данных, метрики, жизненный цикл моделей. Это база, без которой невозможно двигаться в DL.
- ML без связи с бизнесом бесполезен. Любая задача начинается с понимания бизнес-проблемы и проверки, нужен ли тут ML или хватит простого бейзлайна.
- Метрика ≠ loss. Loss — для обучения, метрика — для оценки качества. В регрессии:
  - MAE — устойчива к выбросам, проста в интерпретации;
  - MSE/RMSE — сильнее штрафуют большие ошибки, но чувствительны к выбросам;
  - R^2 — показывает, насколько модель лучше среднего предсказания;
  - MAPE/sMAPE/WAPE/MdAPE — удобны для бизнеса, но нестабильны при малых значениях таргета.;
  - MAE / MSE / Huber loss / quantile loss — популярные функции потерь для задачи классификации.
- В классификации:
  - confusion matrix — основа всех метрик классификации;
  - основные метрики качества классификации с разной логикой: accuracy, precision, recall, F-beta score, F1-score, ROC-AUC;
  - log loss / cross-entropy — популярные функции потерь для задачи классификации;
  - micro-, macro-, weighted-усреднения используются для оценки качества многоклассовых моделей классификации;
  - Hamming score — метрика для многометочной классификации.
- Смысл оценки модели в сравнении. Любую модель нужно сравнивать с бейзлайном и интерпретировать результат в контексте задачи.